Universidade do Minho
Escola de Engenharia

Tiago Manuel Queirós Barata

Sistema operativo orientado às ontologias

junho 2024

Universidade do Minho
Escola de Engenharia

Tiago Manuel Queirós Barata

Sistema operativo orientado às ontologias

Dissertação de Mestrado
Mestrado em Engenharia Informática

Trabalho efetuado sob a orientação de
José João Almeida

junho 2024

Direitos de Autor e Condições de Utilização do Trabalho por Terceiros

Este é um trabalho académico que pode ser utilizado por terceiros desde que respeitadas as regras e boas práticas internacionalmente aceites, no que concerne aos direitos de autor e direitos conexos.

Assim, o presente trabalho pode ser utilizado nos termos previstos na licença abaixo indicada.

Caso o utilizador necessite de permissão para poder fazer um uso do trabalho em condições não previstas no licenciamento indicado, deverá contactar o autor, através do RepositóriUM da Universidade do Minho.

Licença concedida aos utilizadores deste trabalho:

CC BY-SA

https://creativecommons.org/licenses/by-sa/4.0/

Agradecimentos

Gostaria de expressar o meu profundo agradecimento ao Professor José João Almeida, orientador desta tese, cujo apoio, orientação e *insight* foram inestimáveis ao longo deste desafiador e enriquecedor percurso académico. A sua experiência, paciência e dedicação foram fundamentais para a realização deste trabalho.

O Professor José João não apenas me guiou com sua vasta experiência académica, mas também me inspirou a perseguir a excelência e a manter uma atitude de curiosidade e inovação. A sua capacidade de simplificar conceitos complexos e o seu entusiasmo contagiante pelo conhecimento foram cruciais para a minha jornada de aprendizagem e desenvolvimento.

Além do suporte académico, sou grato pelo encorajamento pessoal e pela confiança que o Professor José João depositou em mim durante os momentos mais desafiantes deste projeto. A sua orientação proativa e conselhos sempre oportunos ajudaram-me a manter o foco e a direção.

Este trabalho também beneficiou imenso dos debates e das sessões de *brainstorming* com ele, onde pude explorar novas ideias e refinar o meu pensamento. A sua abordagem colaborativa no processo de orientação permitiu-me desenvolver habilidades de pesquisa autónomas que serão valiosas na minha futura carreira académica e profissional.

Por tudo isso e muito mais, minha sincera gratidão ao Professor José João Almeida, por ser uma fonte de inspiração e um verdadeiro mentor. Este trabalho é um testemunho do seu compromisso em cultivar estudantes não apenas como académicos, mas como pensadores inovadores e colaboradores significativos tanto para a área de Engenharia de Linguagens, como para a Informática.

Obrigado, Professor, por tudo.

Declaração de Integridade

Declaro ter atuado com integridade na elaboração do presente trabalho académico e confirmo que não recorri à prática de plágio nem a qualquer forma de utilização indevida ou falsificação de informações ou resultados em nenhuma das etapas conducente à sua elaboração.

Mais declaro que conheço e que respeitei o Código de Conduta Ética da Universidade do Minho.

Universidade do Minho, Braga, junho 2024

Tiago Manuel Queirós Barata

Resumo

Esta tese apresenta o desenvolvimento e a exploração de um sistema operativo baseado em ontologias (OBOS), que integra as funcionalidades e a simplicidade das ferramentas inspiradas em Unix com o ambiente de dados estruturados das ontologias. A principal motivação por trás deste sistema é melhorar a gestão e manipulação de dados ontológicos nas humanidades digitais e outros domínios onde a representação estruturada do conhecimento é crucial. O OBOS aproveita operações pequenas e simples — semelhantes aos comandos Unix — que podem ser combinadas para realizar tarefas complexas, aderindo assim à filosofia Unix de "fazer bem uma coisa".

Elementos-chave do sistema incluem a verificação da integridade das ontologias, a extração de propriedades e tipos envolvidos em ontologias, e a conversão de diversos formatos de dados em ontologias para facilitar o arquivamento digital e a gestão de dados. Além disso, o OBOS introduz um editor de ontologias projetado para simplificar as interações dos utilizadores com estruturas ontológicas complexas, fornecendo ferramentas para uma pesquisa eficaz, edição e visualização de dados.

A arquitetura do sistema permite expansões modulares e interoperabilidade com formatos de dados e sistemas existentes, tornando-o uma ferramenta versátil para ambientes académicos e profissionais. Integrando funcionalidades como pesquisa baseada em ontologias, edição e conversão de saída (para formatos como HTML e PDF), o OBOS pretende transformar a forma como as ontologias são utilizadas, tornando-as mais acessíveis e eficazes para os utilizadores de diversas áreas.

Palavras-chave sistema operativo orientado às ontologias, humanidades digitais, filosofia Unix, representação do conhecimento, integração de dados, gestão de ontologias

Abstract

This thesis presents the development and exploration of an Ontology Based Operating System (OBOS), which integrates the functionalities and simplicity of Unix-inspired tools with the structured data environment of ontologies. The primary motivation behind this system is to enhance the management and manipulation of ontological data in digital humanities and other domains where structured knowledge representation is crucial. The OBOS leverages small, simple operations—akin to Unix commands—that can be combined to perform complex tasks, thus adhering to the Unix philosophy of "doing one thing well."

Key elements of the system include the verification of ontology integrity, the extraction of properties and types involved in ontologies, and the conversion of various data formats into ontologies to facilitate digital archiving and data management. Furthermore, OBOS introduces an ontology editor designed to simplify user interactions with complex ontological structures, providing tools for effective data searching, editing, and visualization.

The system's architecture allows for modular expansions and interoperability with existing data formats and systems, making it a versatile tool for academic and professional environments. By integrating functionalities such as ontology-based searching, editing, and output conversion (to formats like HTML and PDF), OBOS aims to transform how ontologies are utilized, making them more accessible and effective for users across various disciplines.

Keywords ontology-oriented operating system, digital humanities, Unix philosophy, knowledge representation, data integration, ontology management

Conteúdo

I Material Introdutório **1**

1 Introdução **2**

1.1 Contextualização e Motivação 2

1.2 Principais objetivos 3

1.2.1 Verificação da integridade da ontologia 5

1.2.2 Extração das properties e tipos envolvidos 6

1.2.3 Conversor de formatos vários para ontologia 6

1.2.4 Editor de ontologias 8

1.2.5 Pesquisa em ontologias 8

1.2.6 Conversores de vários formatos para ontologia 8

1.2.7 Criação de templates 10

1.3 Estrutura Geral do Documento 10

1.3.1 Parte I: Material Introdutório 11

1.3.2 Parte II: OBOS: Sistema Operativo Orientado às Ontologias 11

2 Estado da arte **12**

2.1 Ontologias 12

2.1.1 Especificação formal 13

2.1.2 SPARQL: Linguagem de consulta para RDF 21

2.2 Inspiração Unix 22

2.2.1 grep 23

2.2.2 vim 24

2.2.3 wc 25

2.3 Ferramentas 26

2.4 Ferramentas 26
2.4.1 Protégé 26
2.4.2 Ferramentas para consulta SPARQL 27
2.4.3 GraphDB 28
2.4.4 Triplestores 29
2.5 Projetos relacionados 31
2.5.1 Web Semântica 31
2.5.2 RegEx 33
2.5.3 Markdown 33
2.5.4 pandoc 34
2.5.5 RDFLib 35
2.5.6 Owlready2 36
2.5.7 Jinja2: Linguagem de templates 37

II OBOS: Sistema Operativo Orientado às Ontologias 39

3 Estrutura do OBOS 40
3.1 Construção da aplicação 40
3.1.1 Kernel 41
3.1.2 Scripts 42
3.1.3 Classes e indivíduos 42
3.1.4 Filtros 42
3.1.5 Conversores 43
3.1.6 Utils 43
3.2 Tabela de comandos 45

4 Kernel 46
4.1 Armazenamento de Dados 47
4.1.1 Armazenamento semi-permanente 47
4.1.2 Armazenamento interno 48
4.2 Funcionamento Base 49
4.2.1 Parse das classes 50

5 Comandos OBOS **51**
5.1 Tradução . . . 51
5.1.1 Módulos implementados . . . 51
5.1.2 OBOSClasses . . . 53
5.1.3 OBOSSignatures . . . 54
5.1.4 OBOSHtml . . . 56
5.1.5 OBOSTemplate . . . 57
5.2 Pesquisa . . . 60
5.2.1 Módulos implementados . . . 61
5.2.2 OBOSGrep . . . 62
5.2.3 OBOSWc . . . 64
5.2.4 OBOSQuery . . . 65
5.2.5 OBOSVisual . . . 66
5.2.6 OBOSData . . . 67
5.3 Edição . . . 68
5.3.1 Módulos implementados . . . 69
5.3.2 OBOSVim . . . 69
5.4 Validação . . . 70
5.4.1 OBOSLint: Verificação da integridade da ontologia . . . 71
5.4.2 Comando de testes de regressão . . . 72
5.4.3 Comando de análise de qualidade de dados . . . 72
5.4.4 Impacto dos comandos de validação . . . 72

6 Exemplos **73**
6.1 OBOSHtml . . . 73
6.1.1 Árvore de classes . . . 73
6.1.2 Classes . . . 75
6.1.3 Indivíduos . . . 76
6.1.4 Propriedades . . . 80
6.2 OBOSSignatures . . . 81
6.3 OBOSTemplate . . . 83
6.4 OBOSGrep . . . 84
6.5 OBOSVim . . . 84

7 Conclusões e Trabalho Futuro **86**

7.1 Conclusões 86

7.2 Trabalho futuro 86

A Diagrama de classes ontologia FYT **94**

B Uso de filtros ontológicos em templates **95**

B.1 Ontologia 95

B.2 Template 97

B.3 Resultado 99

Lista de Figuras

1 Diagrama representativo do OBOS 5

2 Hierarquia da classe de termos 35

3 Diagrama da estrutura aplicacional OBOS 40

4 Diagrama de classes do kernel do SO 46

5 Diagrama da Estrutura Árvore 48

6 Exemplo de uma Classe - Azul 56

7 Exemplo de um Indivíduo - Vermelho 57

8 Exemplo de uma Propertie - Verde 57

9 Diagrama de hierarquia de classes 74

10 Representação das classes no HTML 75

11 Representação das classes no HTML com o *inverseOf* 76

12 Representação dos indivíduos no HTML 77

13 Conteúdo multimédia no HTML 78

14 Representação dos indivíduos no HTML com TEI 79

15 Representação de MD em HTML 80

16 Representação das propriedades no HTML 81

Lista de Tabelas

1 Tabela de comandos OBOS 45

Listings

2.1 Exemplo de formatação Turtle 14
2.2 Exemplo de prefixos de uma ontologia em Turtle 15
2.3 Exemplo da declaração das classes de uma ontologia em Turtle 16
2.4 Exemplo da declaração das relações de uma ontologia em *Turtle* 18
2.5 Regras de inferência 18
2.6 Exemplo da declaração dos indivíduos de uma ontologia em Turtle 19
2.7 Exemplo de uma consulta SPARQL 22
2.8 Exemplo do grep 23
2.9 Exemplo do vim 25
2.10 Exemplo simples do wc 25
2.11 Exemplo complexo do wc 26
2.12 Exemplo de template Jinja2 38
3.1 Excerto do ficheiro de configurações 43
3.2 Acesso ao ficheiro de configurações 45
5.1 Exemplo de output 53
5.2 Exemplo de output 54
5.3 Exemplo de template 60
6.1 Ontologia exemplo 73
6.2 Taxonomia OBOSHtml 73
6.3 Comando OBOS para escolher a profundidade 74
6.4 Taxonomia de classes com definição da profundidade 74
6.5 Comando OBOS para escolher o topo da árvore de classes 74
6.6 Taxonomia de classes com definição do topo 74
6.7 Ontologia exemplo com campo *inverseOf* 76
6.8 Ontologia exemplo com conteúdo multimédia 77

6.9 Excerto em TEI da ontologia ETL 78
6.10 Excerto em MD da ontologia ETL 79
6.11 *Output default* do 5.1.3 81
6.12 *Output* em JSON 81
6.13 *Output* em YAML 82
6.14 *Output* em *Turtle* 82
6.15 Resultado da pesquisa por “avo” 84
6.16 Resultado da pesquisa por “avo” como sujeito 84
6.17 Edição dos triplos de “avo” 85
6.18 *Output* da edição 85
6.19 Ontologia editada 85
B.1 Ontologia exemplo 95
B.2 Template exemplo 97

Acrónimos

CSS *Cascading Style Sheets*.

DALF Digital Archive of Letters in Flanders.

ETL Eurico Thomaz de Lima.

HTML *Hypertext Markup Language*.

JSON *JavaScript Object Notation*.

MD Markdown.

OBOS *Ontology Based Operating System*.

OWL *Web Ontology Language*.

PDF *Portable Document Format*.

RDF *Resource Description Framework*.

RDFS *Resource Description Framework Schema*.

SO Sistema Operativo.

SPARQL *SPARQL Protocol and RDF Query Language*.

SQL Structured Query Language.

TEI *Text Encoding Initiative*.

TIC Tecnologias da Informação e da Comunicação.

XML *Extensible Markup Language*.

Parte I

Material Introdutório

Capítulo 1

Introdução

Existem muitas coisas que se podem fazer com ontologias, e só o trabalho de escrever uma ontologia poderia ser tema para uma dissertação. Este trabalho procura ajudar no desenvolvimento, visualização e oferecer ferramentas que ajudem na gestão de ontologias já existentes.

Este capítulo estabelece o cenário para a dissertação, introduzindo ontologias dentro do contexto das humanidades digitais, destacando a sua relevância em diversos campos. Além de discutir a motivação por de trás do seu estudo, este capítulo delineia os principais objetivos do projeto, que incluem a melhoria da eficiência na gestão de ontologias e o desenvolvimento de ferramentas avançadas para facilitar processos relacionados. A introdução serve como um ponto de partida para o leitor entender a importância do *sistema operativo* orientado às ontologias, mostrando como se pode transformar a maneira como as ontologias são aplicadas em contextos práticos e académicos.

1.1 Contextualização e Motivação

Na área das humanidades digitais, o uso de ontologias pode ser bastante útil, não só para a criação de simples bases de dados textuais, como para a preservação dos dados de espólios antigos. Neste sentido, utilizando estas ferramentas, consegue-se a representação de conhecimento de forma estruturada e sistematizada, o que facilita a análise e a interpretação de grandes conjuntos de dados. Além disso, as ontologias também podem ser utilizadas para criar sistemas de recuperação de informação e sistemas de inteligência artificial que possam auxiliar os pesquisadores nas suas investigações Presutti and Blomqvist [2013].

Apesar de todas estas vantagens na utilização de ontologias, há também desvantagens inerentes ao seu uso que não posso deixar de referir. Os ficheiros de ontologias são bastante difíceis de escrever, pois a *sintax* deste tipo de ficheiros não é nada intuitiva. Se estivermos a falar de ontologias no formato *N3*[1]

[1] Formato simples de ontologia

é facilmente legível o que está escrito, contudo é bastante trabalhoso fazer o trabalho de escrita desse mesmo ficheiro. Uma outra dificuldade é a visualização destes ficheiros. Uma ontologia facilmente chega às milhares de linhas de informação, tendo a possibilidade de não haver nenhuma ordenação ou forma coerente de apresentar os triplos. Desta forma torna muito mais difícil a leitura e a procura de informação neste tipo de ficheiros Allemang and Hendler [2011]. Até para *debug* torna-se bastante complexo porque não há regras que estabeleçam a estrutura de um documento deste tipo Noy and McGuinness [2001].

Como são simples de escrever os triplos, e a informação é bastante simplificada quando se escreve uma ontologia, torna-se bastante útil e versátil para resolver problemas concretos, criar arquivos, espólios e repositórios em formato digital, entre muitas outras situações. As humanidades digitais conseguem tirar bastante partido das ontologias, e simplificar bastante trabalho que teria de ser feito manualmente Siemens and Unsworth [2016].

Um sistema operativo orientado às ontologias pode ser útil porque as ontologias podem fornecer uma estrutura de dados consistente e bem definida para o sistema operativo. Isso pode ajudar a garantir que os dados armazenados pelo sistema operativo estejam organizados de maneira coerente e possam ser facilmente compreendidos e utilizados pelos utilizadores Smith [2003]. As ontologias também podem ser úteis para a criação de interfaces de utilizador mais intuitivas, já que elas fornecem uma estrutura de dados com a qual os utilizadores podem familiarizar-se e interagir Guarino [1998]. Além disso, as ontologias podem ser usadas para a criação de sistemas de recomendação e para a realização de buscas mais precisas, já que elas fornecem uma estrutura de dados que pode ser utilizada para entender o contexto em que os dados estão sendo usados Berners-Lee et al. [2001].

1.2 Principais objetivos

Para ajudar a combater todas as desvantagens enumeradas na utilização de ficheiros de ontologias, aliado às humanidades digitais, vou falar do projeto OBOS[2] que a partir de operações pequenas e simples, permite juntar várias operações e criar algo mais complexo. Este sistema terá por inspiração a arquitetura dos sistemas operativos *UNIX* e os seus comandos, permitindo o uso destas funcionalidades como filtros, de modo a poder encadeá-las com pipes, compondo assim funções mais complexas a partir do simples, dentro do possível Raymond [2003]. Além disso, num futuro breve, está prevista a disponibilização livre da ferramenta através de um módulo PyPI, pois estas ferramentas serão criadas maioritariamente recorrendo

[2] Explicado no capítulo 3

à linguagem de programação *Python*, e a alguns módulos disponíveis no repositórios(*pip*), tais como *rdflib* e *owlready* Miller and Ranum [2004].

Segue agora uma lista das funcionalidades que se pretendem desenvolver:

1. **Popular** ontologias a partir de um conjunto de recursos / formatos.
2. **Edição** de partes específicas das ontologias.
3. **Pesquisa** de informação específica nas ontologias.
4. **Criação** de vistas tais como HTML, PDF, dicionários, entre outros.
5. **Criação** de templates ontológicos para facilitar a criação de vistas e exportações.
6. **Construção de aplicações web simples** baseadas em ontologias Berners-Lee et al. [2001].

As ontologias desempenham um papel crucial na estruturação de bases de conhecimento, servindo como um esquema formal para a organização e categorização de informações dentro de um domínio específico. Uma base de conhecimento utiliza ontologias para definir explicitamente a estrutura dos dados, incluindo as classes, as propriedades e as relações entre entidades, o que facilita a inferência de novas informações e suporte a consultas complexas e análises de dados. Por exemplo, ontologias permitem a implementação de sistemas de informação mais inteligentes, onde máquinas podem processar e entender os dados de forma semântica, como demonstrado por Allemang e Hendler Allemang and Hendler [2011]. Além disso, a inter-relação entre ontologias e bases de conhecimento é fundamental para o avanço da Web Semântica, permitindo a integração e interoperabilidade entre diferentes sistemas e contextos, conforme descrito por Berners-Lee, Hendler, e Lassila Berners-Lee et al. [2001]. Portanto, a utilização de ontologias é essencial não apenas para a modelação do conhecimento, mas também para a eficácia das operações computacionais e a precisão das decisões baseadas em dados semânticos.

No fundo o que se pretende é um canivete-suíço para ontologias.

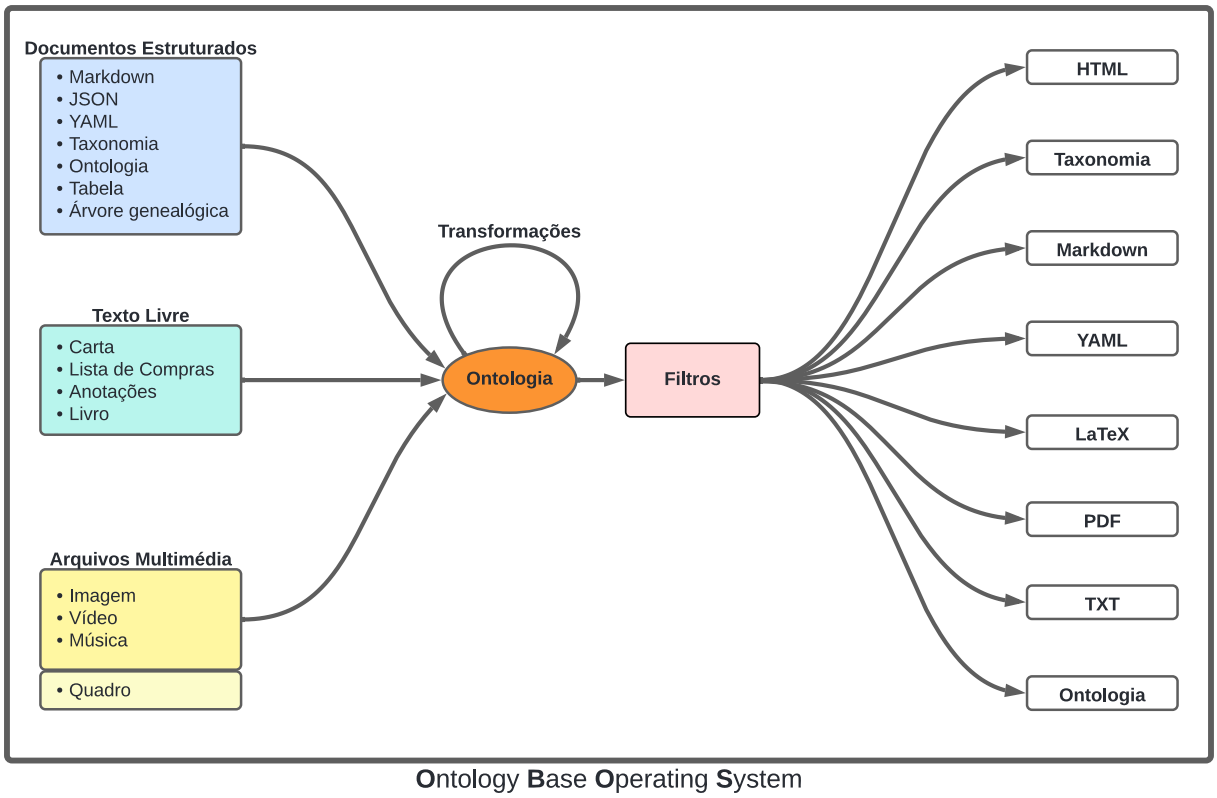


Figura 1: Diagrama representativo do OBOS

Dadas estas funcionalidades e arquitetura, segue uma descrição de possíveis ideias a aplicar.

1.2.1 Verificação da integridade da ontologia

A integridade de uma ontologia é crucial para garantir as suas funcionalidades e a correção dos dados representados. A verificação da integridade envolve a análise das *properties*, das classes e das relações estabelecidas para assegurar que não existem inconsistências ou erros lógicos Staab and Studer [2009].

Assim, é fundamental realizar testes e validações regulares, que podem incluir a verificação de redundâncias, a deteção de ciclos, a análise da cobertura do domínio e alcance, e a conformidade com as regras de inferência. Estes testes podem ser executados manualmente ou através de ferramentas automatizadas, que facilitam a deteção de problemas complexos Gomez-Perez et al. [2004].

Através destes procedimentos, procura-se não apenas manter a consistência lógica interna da ontologia, mas também melhorar a sua eficiência enquanto ferramenta de organização e interpretação de dados em sistemas complexos. O objetivo final é garantir que a ontologia seja mais robusta e confiável para todos os utilizadores e contextos aplicáveis.

1.2.2 Extração das properties e tipos envolvidos

As assinaturas de uma ontologia, ou as *properties* são o que dão estrutura e o que permite a boa representação da informação Noy and McGuinness [2001]. São as relações entre sujeitos e objetos.

Deste modo, pretende-se extrair essas assinaturas em vários formatos, como JSON, *YAML* ou apenas apresentado como texto indentado Klyne and Carroll [2004]. Independentemente da sua representação, é crucial obter toda a informação relativa às assinaturas, como o domínio, o alcance e relações inversas.

Novamente, o objetivo é simplificar a leitura e procura de informações nos documentos textuais de ontologias, possibilitando um *debug* mais simples.

1.2.3 Conversor de formatos vários para ontologia

Uma ontologia é, por definição, uma representação do mundo real. No domínio das Humanidades Digitais, os nosso casos de estudo são ricos em documentos heterogéneos, existindo um passado rico em formatos e modos de transcrição, catalogação e classificação. Quanto maior for a facilidade de fazer essa representação, mais fácil será o tratamento de espólios e criação de arquivos digitais Heath and Bizer [2011]. Neste sentido, pretende-se fornecer ao utilizador um leque de ferramentas que, dado um conjunto de ficheiros variados, possa gerar ontologias, ou através dos metadados, ou de outras informações contidas nos ficheiros.

Seguem alguns pequenos exemplos do que se pretende.

Conversor de imagens para ontologia

Dada uma ou várias imagens, pretende-se gerar um ficheiro de ontologia que descreva a mesma Hyvönen [2009]. Uma imagem pode conter vários campos descritivos além do que é visível, por exemplo:

- Título
- Formato
- Tamanho
- Legenda
- Autor
- Dimensão
- Data

Conversor de cartas para ontologia

As cartas, tal como as imagens têm muita coisa que pode ser aplicada numa ontologia Shadbolt et al. [2006]:

- Remetente
- Destinatário
- Tamanho/formato do papel
- Selo
- Informação do envelope
- Cabeçalho
- Transcrição

Um bom exemplo de uma ontologia com cartas é a dissertação sobre o espólio de Eurico Tomás de Lima Rego [2022], demonstrando não só a utilidade de uma ontologia, como toda a informação que pode ser retirada de uma carta e preservada em formato digital.

Conversor de livros para ontologia

Os livros são uma outra grande fonte de informação, de onde podemos extrair várias informações e guardá-las numa ontologia Berners-Lee et al. [2001].

- Autor
- Título
- Subtítulo
- Editora
- Edição

1.2.4 Editor de ontologias

As ontologias podem ser verdadeiramente extensas, sendo muito fácil atingirem os milhões de linhas. Um dos grandes desafios é saber onde editar, o que procurar, e perceber muitas vezes onde está o erro que se procura há imenso tempo, no meio de tanto texto Musen [2012].

O editor que irá ser desenvolvido procura simplificar a vida ao utilizador, de modo a que, além da facilidade em fazer edições no sítio certo, seja fácil navegar pelo ficheiro, por muito desorganizado que tenha sido criado d'Aquin and Motta [2008].

1.2.5 Pesquisa em ontologias

Toda a gente percebe a importância de um bom sistema de pesquisa. Em ontologias, por vezes temos os identificadores que aglutinam várias palavras, usando os tradicionais *camelCase*, *snake_case* e outros critérios de abreviaturas , e a procura com um simples `ctrl+f` não é nada intuitiva Alani et al. [2003].

Para colmatar este problema, procura-se criar um sistema de pesquisa que ultrapasse esta barreira e possibilite o utilizador de pesquisar mesmo sem saber em concreto o que procurar, através de um sistema versátil e adaptável às necessidades Bechhofer et al. [2009].

A pesquisa em ontologias e bases de conhecimento enfrenta desafios significativos devido à complexidade inerente das estruturas de dados e à variabilidade dos domínios modelados. Um dos principais desafios é a heterogeneidade semântica, onde diferentes ontologias podem representar conceitos similares de formas diferentes, complicando a interoperabilidade e a integração de dados entre sistemas Shvaiko and Euzenat [2005]. Além disso, a ambiguidade na definição dos termos dentro das ontologias pode levar a interpretações erróneas, exigindo um trabalho meticuloso de normalização e alinhamento Noy [2004]. As ontologias são frequentemente extensas e complexas, o que torna a navegação e a pesquisa de informações específicas um processo que exige não apenas ferramentas computacionais avançadas mas também um profundo entendimento do domínio por parte dos utilizadores Hitzler et al. [2009]. Portanto, a pesquisa em ontologias não é apenas uma questão técnica de manipulação de dados, mas também um desafio de compreensão e interpretação de conceitos complexos e interligados.

1.2.6 Conversores de vários formatos para ontologia

Da mesma maneira que existem informações guardadas em ontologias, esses ficheiros não são fáceis de analisar e fazer uma apresentação, por exemplo. Com o objetivo de ajudar a mostrar ontologias, pretende-se criar alguns conversores que permitam ao utilizador visualizar e converter as ontologias para

outros formatos mais “simpáticos” Heath and Bizer [2011].

Este tipo de conversores permitem:

1. tornar as ontologias mais úteis.
2. ajudam à validação por humanos.
3. fomentar a colaboração.

Conversão de ontologias para HTML

Descrevo a criação de HTML a partir de ontologias, mostrando uma visão interligada sem a necessidade de servidores com as seguintes funcionalidades:

1. Uma vista de síntese que mostre a lista de classes existentes, e a cardinalidade de indivíduos por classes, transitivos e diretos Hyvönen [2009].
2. Toda a informação contida na ontologia, com a possibilidade de pesquisa e navegação d’Aquin and Motta [2008].
3. Possibilidade de definir *templates* para otimização de visualização de certas classes, como cartas, pessoas, entre outras.
4. Integração de CSS permitindo uma visualização mais clara, fazendo a distinção por cores do que são classes e do que são elementos, bem como a clara apresentação de literais tipados (como XML ou TEI) Clark [1999].

O objetivo principal desta funcionalidade é, numa única página HTML conter toda as informações da ontologia, de modo a que possa ser navegável, facilitando a vista na ótica do utilizador, permitindo procurar mais facilmente erros, ou apresentar dados. Além disto, seria possível efetuar pesquisa, diminuindo a quantidade de informação apresentada.

Conversor de ontologias para PDF

Parecido com o HTML, os ficheiros PDF podem ser navegáveis e facilmente apresentáveis ao utilizador. Gerar relatórios através de uma ontologia, quer sejam totais ou parciais, é algo que pode ser bastante útil numa apresentação Berners-Lee et al. [2001].

Desta foram, a ideia é colocar toda a informação contida na ontologia num documento que permita o utilizador apresentar um documento deste tipo sem nenhuma preocupação, pois o formato de escrita será legível e perfeitamente articulado com toda a inforamção contida na ontologia.

Conversor de ontologias para Markdown

Além de navegável, o Markdown permite a criação de ficheiros `README` bastante úteis na informática e em muitas outras áreas. Esses ficheiros têm uma escrita muito simples e uma visualização bastante apelativa e muito intuitiva MacFarlane [2013].

Uma vantagem do markdown é a biblioteca PANDOC que permite a conversão de ficheiros deste tipo para imensos formatos diferentes, o que os torna bastante versáteis.

Conversores de ontologia através de template

Todos os conversores referidos nas secções anteriores são bastante úteis e versáteis para o utilizador, no entanto se apenas existirem esses, apesar da utilidades, dá pouca flexibilidade ao utilizador. Deste modo, pretende-se desenvolver uma ferramenta que dê esta flexibilidade ao utilizador. Implica algum trabalho e engenharia de *templates*, mas torna estas conversões mais robustas Mendelsohn [2009].

Estes templates poderão usar filtros jinja2, implementados no OBOS, de modo a facilitar a navegação nas ontologias. Esses filtros serão detalhados mais à frente.

1.2.7 Criação de templates

Para facilitar a escrita de ontologias, serão criados *templates* que ajudem o utilizador a perceber as fases e os processos de escrita das ontologias, disponibilizando alguns campos pré-preenchidos permitindo fazer as alterações necessárias.

Estes *templates* permitirão ainda a criação de ontologias através de certas estruturas de dados onde o utilizador passa os dados e os *templates* geram a ontologia pretendida, sem ter problemas de entender a escrita em linguagens de ontologia.

Aliado a isto, haverão ainda *templates* que gerem todos os outros ficheiros. Por exemplo, os ficheiros HTML serão gerados através de uma estrutura fixa de *template* em Jinja2 2.5.7.

Serão ainda criadas funções Jinja2 que servirão para navegar a ontologia, possibilitando ao utilizador a definição de *templates* personalizados.

1.3 Estrutura Geral do Documento

Esta dissertação está organizada em duas partes principais. Cada uma compreende vários capítulos que discutem diferentes aspetos do tema central: o sistema operativo orientado às ontologias.

1.3.1 Parte I: Material Introdutório

- **Capítulo 1: Introdução** Neste capítulo discute a relevância e aplicabilidade das ontologias em diferentes contextos, especialmente em humanidades digitais. Descrevem-se ainda os objetivos específicos do projeto, como melhorar a eficiência na gestão de ontologias e desenvolver ferramentas para facilitar esse processo.

- **Capítulo 2: Estado da Arte** Neste capítulo descreve-se o conceito e especificação de ontologia, um bocado da história do Unix bem como alguns comandos que serviram de inspiração para este trabalho, ferramentas que foram úteis ou serviram de inspiração ao longo do desenvolvimento, projetos onde são utilizadas ontologias e módulos python que foram utilizados para o desenvolvimento do OBOS.

1.3.2 Parte II: OBOS: Sistema Operativo Orientado às Ontologias

- **Capítulo 3: Estrutura** Neste capítulo apresenta-se a estrutura aplicacional, bem como a ligação entre os diversos componentes, justificando as escolhas feitas na construção da aplicação e uma lista todos os comandos criados para o OBOS, resumindo a sua função e o grupo a que pertence.

- **Capítulo 4: Kernel** Descreve a implementação e funcionalidades do kernel do sistema operativo, focando em como é feita a gestão de dados e processos.

- **Capítulo 5: Comandos** Apresenta os comandos específicos desenvolvidos para o sistema operativo, como funcionam e como é feita a interação utilizador - sistema operativo.

- **Capítulo 6: Exemplos** Fornece exemplos práticos de como o sistema pode ser utilizado, ilustrando casos de uso reais.

- **Capítulo 7: Conclusões e Trabalho Futuro** Sumariza os resultados alcançados e discute direções futuras para pesquisa e desenvolvimento do sistema.

Cada capítulo é projetado para construir sobre o conhecimento introduzido nos capítulos anteriores, assegurando uma compreensão profunda e sistemática do tema em estudo.

Capítulo 2

Estado da arte

Neste capítulo começa-se por fazer uma clarificação de conceitos, especificações, concretizar e discutir ontologias e as suas partes e a filosofia e metáfora dos filtros linux. De seguida mostrarei projetos e módulos que me possam ajudar no futuro para o desenvolvimento.

Ter uma base sólida e concisa para crescer e criar algo novo, como dizia Isaac Newton na sua célebre frase "Se eu vi mais longe, foi por estar sobre ombros de gigantes.". [Newton, 1675]

2.1 Ontologias

Considere-se a seguinte situação ligada a um arquivo familiar: "No dia 3 de maio, o João enviou uma carta à Maria a dizer que ia para o Brasil.". Numa abordagem de especificação através de ontologias, podemos descrever esta situação através do seguinte conjunto de triplos:

- (João - é uma - pessoa)
- (Maria - é uma - pessoa)
- (C1 - é uma - carta)
- (João - escreveu - C1)
- (Maria - recebeu - C1)
- (C1 - tem como transcrição - "Querida Maria, vou para o Brasil")

Com base no exemplo anterior, podemos extrair os indivíduos, as classes e as relações da ontologia. Os indivíduos são 'João', 'Maria' e 'C1', as classes são 'pessoa' e 'carta' e as relações são 'é uma', 'escreveu', 'recebeu' e 'tem como transcrição'. Chamaremos ontologia ao conjunto de classes, indivíduos, relações e triplos que relacionam os elementos entre si Gruber [1993].

Desta forma conseguimos perceber a noção básica de ontologia. Esta foi dada inicialmente pela filosofia na antiguidade clássica, como a representação do mundo real. No entanto, as ontologias no contexto informático são algo bastante simples de perceber; resumem-se a triplos de informação, que são facilmente interpretados por um processador textual Guarino [1998].

Apesar disto, podemos ver que uma simples frase pode dar origem a vários triplos. Não deixa se ser simples, mas torna-se muito extenso. As ontologias são criadas a partir de indivíduos que podem ser qualquer conceito que se queira representar. Estes indivíduos serão agrupados em classes, por tipologia, espécie ou grupo. Partindo destes elementos, são formados os triplos, compostos por sujeito, predicado e objeto Smith [2003].

Conseguimos ver aqui uma enorme ajuda para as humanidades digitais, pois as ontologias são modelos conceituais que visam a representação de conhecimento e informação de forma estruturada e sistematizada. Na informática, as ontologias são utilizadas como ferramentas de representação de conhecimento para a construção de sistemas de informação, como bases de dados e sistemas de recuperação de informação Berners-Lee et al. [2001]. As humanidades digitais são uma área de estudo que se ocupa da utilização de TIC para o estudo e a interpretação de fenómenos culturais, sociais e históricos. Assim sendo, as ontologias são uma ferramenta importante nas humanidades digitais, pois permitem a representação de conhecimento de forma estruturada e sistematizada, o que facilita a análise e a interpretação de grandes conjuntos de dados. Além disso, as ontologias também podem ser utilizadas para criar sistemas de recuperação de informação que possam auxiliar os investigadores nos seus trabalhos Hendler [2002].

2.1.1 Especificação formal

Consideremos uma ontologia com a seguinte estrutura:

- Prefixos
- Declaração das classes
- Declaração das relações
- Descrição dos indivíduos

Além deste tipo estruturado de dados, há algumas regras que são necessárias seguir para a interpretação de um ficheiro como ontologia. No caso de ficheiros *Turtle* (como utilizados nos exemplos seguintes),

cada triplo tem de terminar com um ponto final. Podem-se declarar vários triplos com o mesmo sujeito, utilizando o ponto e vírgula e vários indivíduos com a mesma relação a vários objetos, separando-os apenas por vírgula Klyne and Carroll [2004].

```
subj pred obj .
subj pred obj1,
            obj2 .
subj pred1 obj1;
        pred2 obj2 .
subj pred1 obj1 ;
        pred2 obj2,
                obj3 .
```

Listing 2.1: Exemplo de formatação Turtle

Prefixos

Os prefixos numa ontologia são abreviações ou atalhos que representam *namespaces* ou URIs (Uniform Resource Identifiers) mais longos, usados para simplificar a declaração e o uso de recursos em ontologias. São fundamentais para gerir e organizar eficientemente os termos dentro de uma ontologia, para evitar repetições desnecessárias de URIs completas ao longo do documento, o que torna a ontologia mais legível e fácil de manter.

Função dos prefixos em ontologias

- Simplificação de URIs: Um prefixo substitui um URI longo por uma etiqueta curta, facilitando a leitura e escrita do código. Por exemplo, em vez de repetir um URI completo como `http://www.example.org/ontology#`, podemos simplesmente usar um prefixo como `ex:` para representar essa base Allemang and Hendler [2011].

- Organização e Clustering de Termos: Prefixos ajudam a agrupar termos relacionados sob um mesmo *namespace*, o que é útil para diferenciar e organizar conceitos de diferentes domínios Noy and McGuinness [2001].

- Reuso de Ontologias: Ao definir prefixos para *namespaces* comuns, como aqueles definidos pelo RDF, RDFS, OWL, e outros vocabulários padrões, facilita-se a integração e o reuso de ontologias e vocabulários existentes Berners-Lee [1999].

- Compatibilidade e Interoperabilidade: O uso de prefixos conforme padrões estabelecidos promove interoperabilidade entre diferentes sistemas e aplicações que utilizam ontologias, pois permite que entidades e conceitos sejam facilmente reconhecidos e interpretados entre diferentes plataformas e contextos Heath and Bizer [2011].

Declaração dos prefixos Em documentos de ontologia, especialmente aqueles escritos em formatos como RDF/XML, Turtle, ou OWL, os prefixos são declarados no início do documento. A declaração associa uma etiqueta curta a uma URI completa, o que permite que a etiqueta seja usada como um substituto compacto da URI nas definições subsequentes Christian Bizer and Berners-Lee [2009].

Exemplo em Turtle A partir do *turtle* a leitura é bastante simples:

```
@prefix : <http://tania/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xml: <http://www.w3.org/XML/1998/namespace> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@base <http://www.semanticweb.org/tania/ontologies/2021/1/my-ontology> .
```

Listing 2.2: Exemplo de prefixos de uma ontologia em Turtle

Neste exemplo, `rdf:`, e `rdfs:` são prefixos que representam respectivamente a ontologia de exemplo, RDF, e RDFS. Isso permite que elementos da ontologia sejam especificados de maneira mais concisa, como `rdf:type`, ou `rdfs:label` Berners-Lee et al. [2001].

Importância prática A definição e uso de prefixos são práticas essenciais para a criação e manutenção de ontologias claras e organizadas. Os prefixos ajudam a evitar erros, facilitam a leitura do código por humanos e sistemas, e permitem que desenvolvedores e investigadores colaborem mais eficientemente, partilhando e reutilizando ontologias de forma consistente e padronizada. Além disso, simplificam a integração de diferentes fontes de dados semânticos, essenciais para a Web Semântica Lassila and Swick [1999].

Classes

As classes são fundamentais na estrutura de uma ontologia, servindo como a espinha dorsal para a organização e categorização de conceitos dentro de um domínio específico. Uma classe em uma ontologia pode ser vista como um conjunto que inclui indivíduos ou instâncias que partilham características ou atributos comuns Smith [2003].

Função das classes As classes proporcionam uma maneira sistemática de agrupar entidades semelhantes sob um mesmo conceito, facilitando a inferência e a consulta de dados. Por exemplo, numa ontologia que modela um evento, uma classe pode representar todos os "Participantes", enquanto outra pode representar diferentes tipos de "Sessões" ou "Atividades" Noy and McGuinness [2001].

Hierarquia de classes Uma característica importante das classes nas ontologias é a capacidade de formar hierarquias. Uma classe pode ser uma subclasse de outra classe, indicando que a subclasse herda características da classe pai. Esta relação hierárquica ajuda na construção de uma base de conhecimento mais rica e na execução de raciocínio lógico sobre os dados Rector [2003].

Declaração de classes em Turtle O Turtle é frequentemente usado para declarar classes em ontologias devido à sua legibilidade e simplicidade. No exemplo abaixo, demonstramos como diferentes classes são declaradas e como elas podem ser relacionadas umas com as outras através de propriedades como `rdf:type` e `rdfs:subClassOf` Beckett [2008].

```
:Assumir_Cargo rdf:type owl:Class ;
               rdfs:subClassOf :Evento .
:Bilhete_Postal rdf:type owl:Class ;
                rdfs:subClassOf :Correspondência .
:Bilhete_Postal_Datilografado rdf:type owl:Class ;
                              rdfs:subClassOf :Bilhete_Postal .
:Carta rdf:type owl:Class ;
       rdfs:subClassOf :Correspondência .
```

Listing 2.3: Exemplo da declaração das classes de uma ontologia em Turtle

Desconstruindo o exemplo acima, podemos ver a seguinte hierarquia de classes:

- Evento
  - Assumir Cargo
- Correspondência
  - Carta
  - Bilhete postal
    - Bilhete Postal Datilografado

Através desta hierarquização consegue-se ter uma maior organização e flexibilidade de conceitos.

Uso Prático das Classes No contexto prático, as classes são usadas para modelar e instanciar dados dentro de uma ontologia, permitindo que aplicações e agentes de software interajam de maneira mais eficaz com esses dados Horrocks et al. [2003].

Relações

As relações constituem o núcleo de qualquer ontologia, desempenhando um papel crucial na ligação dos conceitos e na formação de estruturas de dados complexas. São fundamentais para criar triplos de informação, que são a base da representação do conhecimento em ontologias Berners-Lee et al. [2001]. Cada relação conecta dois conceitos ou entidades, de modo a descrever como se relacionam um com o outro dentro do domínio modelado pela ontologia.

Função das relações As relações permitem que a informação contida numa ontologia adquira significado e contexto. Sem relações, a ontologia seria composta por uma coleção de termos e classes desconexos, sem nenhuma interação ou relação. As relações ajudam a definir a natureza das interações entre classes e indivíduos, enriquecendo a ontologia com informações úteis para processamento e consulta Allemang and Hendler [2011].

Declaração de relações em Turtle A sintaxe Turtle é usada para declarar relações em ontologias devido à sua simplicidade e eficácia Klyne and Carroll [2006].

Dentro de uma ontologia, as propriedades são usadas para definir relações entre indivíduos ou entre indivíduos e valores de dados. Existem dois tipos propriedades:

- **Object Properties**: Ligam um indivíduo a outro indivíduo. Por exemplo, a propriedade ':remetente' conecta um cartão a uma pessoa.

- **Datatype Properties**: Ligam um indivíduo a um valor de dado (literal). Por exemplo, a propriedade ':data' associa um cartão a uma data específica, que é representada como um literal.

```
:about a owl:ObjectProperty;
       owl:inverseOf :see_also .
:remetente a owl:ObjectProperty.

:biografia a owl:DatatypeProperty.
:data a owl:DatatypeProperty.

:C002 :remetente :Ludovina_Frias_de_Matos ;
      :data "1965-10-21"^^xsd:date .
```

Listing 2.4: Exemplo da declaração das relações de uma ontologia em *Turtle*

Uma sintaxe semelhante mas ainda mais simples que *Turtle* é N3.

Relações e Regras de Inferência As relações também são essenciais para aplicar regras de inferência dentro de uma ontologia. Estas regras permitem que sistemas de conhecimento deduzam novas informações a partir dos dados existentes, usando lógicas formais baseadas em como as entidades estão relacionadas Patel-Schneider et al. [2004].

```
(a, subClasse, b)   =>  a e b são Classe
(a, tipo, classe)   =>  a é uma Classe
(a, domínio, b)     =>  a é uma Property, b é uma Classe
(a, range, b)       =>  a é uma Property, b é uma Classe
    (a, range, b) AND b in SympleTypes => a é uma DataProperty
    (a, range, b) AND b not in SympleTypes => a é uma ObjectProperty

(a, inverso, b)     =>  a e b são Object Properties

(a, tipo, b)        =>  a é um Individuo, b é uma Classe
```

Listing 2.5: Regras de inferência

Este conjunto de declarações e regras mostra como as relações são usadas para construir e manipular o esquema de uma ontologia, permitindo que ela represente com precisão as complexidades de um

domínio específico. As relações são, portanto, vitais para a utilidade e a funcionalidade das ontologias em aplicações de Web Semântica e sistemas inteligentes.

Indivíduos

Os indivíduos de uma ontologia são as entidades concretas ou instâncias sobre as quais a informação é recolhida e organizada dentro de uma estrutura semântica. São os elementos fundamentais que povoam uma ontologia e sobre os quais as propriedades e relações são aplicadas para descrever e conectar dados de maneira significativa Horrocks et al. [2003].

Caracterização dos indivíduos Os indivíduos são os sujeitos das ontologias, representando objetos do mundo real ou conceitos que são de interesse no domínio que a ontologia visa modelar. Eles podem ser pessoas, locais, objetos ou qualquer outra entidade específica. Os indivíduos são descritos dentro da ontologia utilizando classes que definam as suas características gerais e propriedades que fornecem detalhes específicos Staab and Studer [2009].

Literais Os **literais** são valores de dados fixos, como strings, números ou datas, que são usados para fornecer informações específicas sobre as características de um indivíduo. São essenciais para adicionar detalhes concretos aos indivíduos numa ontologia, permitindo uma rica descrição semântica e detalhada do domínio modelado Allemang and Hendler [2011].

Declaração de Indivíduos em Turtle A sintaxe Turtle é usada para declarar indivíduos, especificando as suas classes e atribuindo valores às propriedades que descrevem as suas características e relações com outros indivíduos Beckett [2008].

No exemplo abaixo, vários indivíduos são declarados juntamente com suas propriedades e relações interligadas.

```
:Ludovina_Frias_de_Matos    a :Pessoa .
:Eurico_Thomaz_de_Lima      a :Pessoa .
:Ângela                     a :Pessoa .

:Porto a :Local .

:C002 a :Cartão ;
      :remetente :Ludovina_Frias_de_Matos ;
      :data "1965-10-21"^^xsd:date ;
```

```
10        :menciona :Porto , :Ângela , :Eurico_Thomaz_de_Lima .
```

Listing 2.6: Exemplo da declaração dos indivíduos de uma ontologia em Turtle

Namespace

As ontologias desempenham um papel crucial na representação de conhecimento em diversas áreas, proporcionando uma estrutura semântica para a descrição e compreensão de domínios específicos. Um elemento fundamental em ontologias é o conceito de *namespaces*, que desempenha um papel vital na organização e distinção de termos e conceitos Berners-Lee et al. [2001].

Definição de Namespaces Os namespaces em ontologias referem-se a mecanismos que permitem a atribuição de identificadores únicos a termos e conceitos. Esses identificadores ajudam a evitar ambiguidades e colisões de termos, promovendo a clareza e a precisão na representação do conhecimento Klyne and Carroll [2006]. Cada namespace atua como um espaço isolado, garantindo a distinção entre termos semelhantes provenientes de diferentes fontes ou contextos.

Organização Hierárquica Os *namespaces* são frequentemente organizados de forma hierárquica, seguindo uma estrutura que reflete a relação entre diferentes conceitos. Essa organização contribui para a modularidade e extensibilidade das ontologias, facilitando a incorporação de novos termos e conceitos sem interferir nos existentes Patel-Schneider et al. [2004]. A hierarquia de *namespaces* também permite uma fácil navegação e compreensão da estrutura global da ontologia.

Evitando Conflitos de Nomenclatura Um dos principais desafios em ontologias é evitar conflitos de nomenclatura, especialmente quando diferentes ontologias são combinadas ou reutilizadas. Os *namespaces* abordam esse desafio fornecendo um contexto exclusivo para cada termo, minimizando a probabilidade de confusão ou ambiguidade Hitzler et al. [2010]. Essa abordagem promove a interoperabilidade entre ontologias, permitindo a integração eficiente de diferentes fontes de conhecimento.

Aplicações Práticas Os *namespaces* têm diversas aplicações práticas em ontologias. Por exemplo, em ontologias que representam domínios específicos, como medicina ou ciência da computação, os *namespaces* ajudam a estruturar e organizar os termos relacionados a esses campos Smith [2003]. Além disso, em ontologias colaborativas, onde contribuidores distintos podem adicionar conceitos, os *namespaces* garantem a harmonia e consistência na representação do conhecimento Motik et al. [2009].

Interconexão de Ontologias A interconexão de ontologias é uma área onde os *namespaces* desempenham um papel crucial. Ao adotar identificadores únicos, os *namespaces* facilitam a integração de diferentes ontologias, promovendo a construção de um conhecimento mais abrangente e interligado Berners-Lee [2006]. Isso é particularmente valioso em contextos onde a colaboração entre múltiplos domínios é essencial.

2.1.2 SPARQL: Linguagem de consulta para RDF

O SPARQL, é uma linguagem de consulta padronizada e poderosa projetada especificamente para consultar dados escritos no formato RDF. É uma das tecnologias chave da Web Semântica e permite aos utilizadores escrever consultas complexas para extrair informações de grandes conjuntos de dados distribuídos que estão formatados em RDF Prud'hommeaux and Seaborne [2008].

Características principais de SPARQL É notável a capacidade do SPARQL em consultar diversas bases de dados RDF ao mesmo tempo, o que permite que as consultas ultrapassem fronteiras de documentos ou de bases de dados. Isto é especialmente útil em ambientes onde dados estão interligados em uma rede semântica global, como a Web Harris [2013].

- **Flexibilidade de consulta:** com SPARQL pode-se realizar consultas que envolvem a procura de padrões específicos de dados, com suporte para uniões, consultas opcionais e até mesmo sub-consultas dentro de uma única consulta.
- **Funcionalidades de filtragem:** Permite o uso de filtros para refinar resultados de consultas, aplicando funções lógicas e operadores matemáticos para extrair apenas os dados relevantes.
- **Suporte a consultas complexas:** Capacidade de realizar consultas sobre várias bases de dados RDF espalhadas pela Internet, sem a necessidade de agrupar os dados num único local central DuCharme [2013].

Sintaxe de Consulta A sintaxe do SPARQL é semelhante à do SQL, o que facilita aqueles que já estão familiarizados com bases de dados relacionais. Uma consulta SPARQL típica consiste num conjunto de cláusulas, incluindo `SELECT`, `WHERE`, `FILTER`, entre outras, que definem o que deve ser devolvido como resultado e quais padrões que devem ser procurados nos dados. No exemplo a seguir está-se a extrair todas as propriedades da ontologia.

```
SELECT ?subject ?predicate ?object
WHERE {
  ?subject ?predicate ?object.
  FILTER (?predicate = <http://example.org/property>)
}
```

Listing 2.7: Exemplo de uma consulta SPARQL

Aplicações de SPARQL As aplicações de SPARQL são vastas, especialmente em campos que dependem da integração de dados e da recuperação de informação de fontes dispersas. É amplamente utilizado em:

- Integração de dados empresariais: Unificação de dados de diferentes silos empresariais para análise e relatórios.
- Pesquisas académicas: Facilita a pesquisa em conjuntos de dados académicos que são mantidos em formatos RDF, o que proporciona uma maneira poderosa de descobrir novas conexões e *insights*.
- Aplicações governamentais: Os governos usam SPARQL para ligar e consultar bases de dados públicas e melhorar a transparência e a governação.

2.2 Inspiração Unix

Os sistemas operativos *Unix* foram desenvolvidos com inspiração base de um outro sistema mais antigo chamado de *Multics*. Este sistema operativo tinha como objetivo fazer muitas tarefas, de grande porte para grandes máquinas. No entanto, o *Unix* teve o objetivo de correr em máquinas simples, sem grandes caraterísticas de hardware Ritchie and Thompson [1974].

Assim, o *Unix* ficou bastante conhecido pela sua filosofia de "faça uma coisa e faça-a bem" que identifica claramente a construção do sistema. Ele é composto por muitos pequenos programas especializados, que juntos constroem operações mais complexas e permitem executar tarefas mais específicas Raymond [2003].

Inspirados por esta filosofia, o OBOS vai ser baseado em pequenas operações elementares, que conjugadas conseguem executar tarefas complexas e mais específicas, de acordo com as necessidades do utilizado.

2.2.1 grep

O comando grep é uma ferramenta poderosa e amplamente utilizada em sistemas operativos baseados em Unix, como o Linux Foundation [2021]. O termo "grep"deriva do comando original "g/re/p"usado no editor de texto ed, onde "re"representa uma expressão regular que descreve um padrão de texto a ser pesquisado e "p"indica a ação de imprimir linhas correspondentes.

A principal função do grep é pesquisar e filtrar linhas de texto em arquivos ou fluxos de entrada, retornando apenas as linhas que correspondem ao padrão especificado. Isso torna o grep uma ferramenta valiosa para tarefas como análise de textos, procura de informações em arquivos de texto extensos e processamento de dados.

Além do padrão básico de busca, o grep suporta expressões regulares, que são padrões mais complexos e flexíveis. Isso permite realizar buscas mais avançadas, incluindo correspondência de padrões específicos, pesquisa de padrões com base em classes de caracteres e muito mais.

Alguns dos principais parâmetros do grep incluem:

- *i (ignore case):* Ignora a diferenciação entre maiúsculas e minúsculas durante a busca.
- *r (recursive):* Realiza a busca recursivamente em diretórios, incluindo todos os subdiretórios.
- *n (line number):* Exibe o número da linha junto com a correspondência encontrada.
- *v (invert match):* Inverte a correspondência, exibindo todas as linhas que não correspondem ao padrão.
- *E (extended regular expression):* Permite o uso de expressões regulares estendidas para padrões mais avançados.

Exemplo de uso básico do grep:

```
grep "padrão" arquivo.txt
```

Listing 2.8: Exemplo do grep

Este comando procurará todas as ocorrências do "padrão"no arquivo.txt e mostra todas as linhas correspondentes.

O grep é uma ferramenta versátil que desempenha um papel crucial na linha de comando do Linux, facilitando a análise eficiente de grandes conjuntos de dados textuais.

2.2.2 vim

O Vim é um editor de texto avançado amplamente utilizado em sistemas operativos baseados em Unix, como o Linux Moolenaar [2021]. Desenvolvido como uma melhoria do editor de texto vi, o Vim (Vi Improved) oferece uma ampla gama de recursos e funcionalidades que o tornam numa escolha popular.

Uma característica distintiva do Vim é seu modo de operação, que inclui modos como o modo de inserção, o modo de comando e o modo de visualização. No modo de comando, os utilizadores podem navegar, editar e executar operações com eficiência usando combinações de teclas, o que diferencia o Vim de editores de texto mais tradicionais.

Principais características e funcionalidades do Vim incluem:

1. Modos de Operação:
    - Modo Normal: Permite a navegação eficiente e a execução de comandos.
    - Modo de Inserção: Facilita a inserção de texto no documento.
    - Modo de Visualização: Permite a seleção de blocos de texto para operações como cópia ou exclusão.
2. Expressões Regulares:
    - Suporta expressões regulares para procura e substituição de padrões em documentos.
3. Divisão de Tela:
    - Possibilita dividir a tela para visualizar e editar diferentes partes do mesmo documento ou múltiplos documentos em simultâneo.
4. Extensibilidade:
    - Oferece suporte a uma variedade de plugins e scripts, permitindo a personalização e extensão das funcionalidades do Vim.
5. Recursos de Edição Avançados:
    - Inclui comandos para copiar, colar, undo, redo, entre outros, proporcionando um ambiente de edição robusto.
6. Modo de Linha de Comando:

- Permite a execução de comandos diretamente do modo de comando, proporcionando maior flexibilidade e controle.

7. Configuração Personalizada:

   - Os usuários podem personalizar extensivamente o ambiente do Vim através de arquivos de configuração.

Exemplo de uso básico do Vim:

```
vim arquivo.txt
```

Listing 2.9: Exemplo do vim

Este comando abrirá o arquivo.txt no Vim, permitindo a navegação, edição e guardar alterações.

2.2.3 wc

O comando wc no Linux é uma ferramenta de linha de comando projetada para contar palavras, linhas e carateres em arquivos de texto Foundation [2021]. O nome "wc", é derivado de "word count", que era sua função original. No entanto, ao longo do tempo, foi evoluindo para fornecer informações mais abrangentes sobre o conteúdo de um arquivo.

Principais características e funcionalidades do wc incluem:

1. Contagem de Palavras, Linhas e Caracteres: O wc fornece contagens precisas de palavras, linhas e caracteres em um arquivo de texto.

2. Opções Adicionais: Além da contagem padrão de palavras, linhas e caracteres, o wc pode ser usado com opções adicionais para contar apenas palavras (-w), apenas linhas (-l), ou apenas caracteres (-c).

3. Suporte a Múltiplos Arquivos: O wc pode receber vários arquivos como argumentos, fornecendo contagens individuais e totais para cada um deles.

Exemplo de uso básico do wc:

```
wc arquivo.txt
```

Listing 2.10: Exemplo simples do wc

Este comando exibirá a contagem de linhas, palavras e caracteres no arquivo.txt. Exemplo de uso com opções adicionais:

```
wc -lwc arquivo1.txt arquivo2.txt
```

Listing 2.11: Exemplo complexo do wc

Este comando mostra o número total de linhas, palavras e carateres para os arquivos arquivo1.txt e arquivo2.txt.

O wc é uma ferramenta eficiente para realizar análises rápidas sobre o conteúdo de arquivos de texto, sendo útil em situações em que é necessário obter informações estatísticas básicas sem a necessidade de abrir ou processar o conteúdo do arquivo.

2.3 Ferramentas

Há várias ferramentas já desenvolvidas para trabalhar com ontologias. Apesar de não serem perfeitas, muitas das funcionalidades existentes permitem facilitar o desenvolvimento desta dissertação, tanto em criação de conteúdo como em testes.

2.4 Ferramentas

Há várias ferramentas já desenvolvidas para trabalhar com ontologias. Apesar de não serem perfeitas, muitas das funcionalidades existentes permitem facilitar o desenvolvimento desta dissertação, tanto em criação de conteúdo como em testes.

2.4.1 Protégé

O Protégé é uma plataforma de software de código aberto para edição, visualização e análise de ontologias. É amplamente utilizado por pesquisadores e profissionais para criar, editar e gerir ontologias Musen [2015].

No entanto, esta aplicação tem algumas limitações, tais como:

1. Suporte limitado para linguagens de ontologia: O Protégé é especialmente projetado para trabalhar com OWL, mas também oferece suporte a outras linguagens de ontologia, como o RDF e o RDFS. No entanto, não oferece suporte a todas as linguagens de ontologia existentes.

2. Dificuldade de uso para utilizadores não familiarizados com ontologias: O Protégé é uma ferramenta de nível avançado que pode ser difícil de usar por aqueles que não estão familiarizados com ontologias ou com linguagens formais de representação do conhecimento Knublauch et al. [2004].

3. Limitado em termos de recursos de visualização: O Protégé oferece alguns recursos de visualização básicos, como diagramas de classes e hierarquias, mas não possui recursos avançados de visualização, como gráficos dinâmicos ou recursos de visualização de dados em tempo real Falconer and Storey [2011].

4. Pouco suporte para a colaboração em tempo real: O Protégé não possui recursos de colaboração em tempo real, o que pode ser um problema para equipes que desejam trabalhar em projetos de ontologia de maneira colaborativa Noy et al. [2006].

Em geral, o Protégé é uma ferramenta útil para aqueles que desejam criar e gerir ontologias, mas pode ter algumas limitações para utilizadores que desejam recursos avançados de visualização ou colaboração em tempo real.

Neste sentido, com o objetivo de melhorar o que vai sendo feito, pretende-se colmatar algumas destas falhas enumeradas do Protégé, oferecendo recursos simples ao utilizador, bem como um software mais leve, sem interface gráfica. Apesar disto, o Protégé será utilizado para fazer alguns testes, *debugs* e confirmação de resultados obtidos com os conjuntos de dados fornecidos ao programa desenvolvido.

2.4.2 Ferramentas para consulta SPARQL

SPARQL é uma linguagem de consulta poderosa usada para interagir com dados armazenados em formatos RDF. Existem várias ferramentas e plataformas que suportam SPARQL, o que permite a realização de consultas complexas sobre grandes conjuntos de dados semânticos. Algumas das ferramentas mais notáveis incluem:

- **Apache Jena Fuseki:** É um servidor de SPARQL para bases de dados RDF. Fuseki suporta consultas SPARQL 1.1, atualizações, gráficos nomeados e toda a gestão de conjuntos de dados RDF. É uma solução robusta e escalável para empresas e investigadores que precisam de uma plataforma de gestão de dados semânticos eficiente Carroll et al. [2011].

- **Virtuoso Universal Server:** Desenvolvido pela OpenLink Software, o Virtuoso é uma plataforma que combina bases de dados, servidor de aplicações e funcionalidades de servidor web. Suporta

SPARQL dentro de um quadro de gestão de dados que integra SQL, XML, e RDF. É conhecido pela sua alta performance e capacidade de escalonamento Erling and Mikhailov [2009].

- **Blazegraph:** Conhecido anteriormente como Bigdata, é um armazenamento de grafos ultra escalável, suportando consultas SPARQL contra bases de dados contendo milhares de milhões de triplos. Blazegraph tem um excelente desempenho tanto em ambientes de máquina única como distribuídos, utilizando GPUs para acelerar as consultas Thompson et al. [2014].

- **Stardog:** Plataforma de banco de dados de grafos que suporta consultas SPARQL, e é projetada para aplicações empresariais. Stardog é único porque oferece suporte a raciocínio inferencial e complexos sobre os dados, tornando-o ideal para projetos que requerem uma análise detalhada de redes de dados interconectadas Pérez et al. [2006].

- **GraphDB:** Uma base de dados de grafos que enfatiza a importância das relações nos dados. GraphDB permite armazenar, organizar e gerir coleções de dados RDF utilizando SPARQL para consultas. É especialmente útil para integrar e gerir grandes volumes de dados dispersos Bishop [2011].

Estas ferramentas oferecem funcionalidades distintas que atendem a diferentes necessidades de projetos de dados semânticos, desde aplicações empresariais até pesquisa acadêmicas, facilitando a exploração e a gestão de grandes conjuntos de dados semânticos.

2.4.3 GraphDB

GraphDB é uma base de dados de grafos otimizada para armazenar e gerir ontologias, oferecendo uma plataforma robusta para a Web Semântica e aplicações de *Linked Data*. Este sistema de gestão de base de dados é construído sobre o modelo RDF, permitindo que os utilizadores criem, armazenem e consultem dados interligados de forma eficaz através de SPARQL, a linguagem de consulta RDF Bishop [2011].

Principais características GraphDB é conhecido por sua capacidade de armazenamento eficiente de grandes volumes de dados semânticos, fornecendo mecanismos avançados de inferência e pesquisa, que facilitam a manipulação e extração de conhecimento complexo a partir de redes semânticas de grandes dimensões e interconectadas Kiryakov et al. [2005]. Algumas das características principais incluem:

- Inferência: GraphDB oferece suporte robusto para regras de inferência, permitindo que os utilizadores definam seus próprios axiomas e regras lógicas que automaticamente enriquecem os dados semânticos armazenados.

- Desempenho escalável: Projetado para alta performance, suporta grandes conjuntos de dados e consultas complexas, tornando-o adequado para ambientes empresariais e de pesquisa que requerem processamento rápido e eficiente de consultas semânticas Corporation [2015].

- Integração e interoperabilidade: Facilita a integração com diversas fontes de dados e outros sistemas de gestão de base de dados, suportando uma variedade de formatos de importação e exportação de dados.

Utilidade com Ontologias A utilização de GraphDB com ontologias permite uma gestão eficiente do conhecimento, oferecendo uma plataforma dinâmica para:

- Gestão de dados conectados: Capacita as organizações a integrar, explorar e analisar dados interrelacionados originários de múltiplas fontes, maximizando assim o valor dos dados através das conexões semânticas Feigenbaum [2007].

- Desenvolvimento de aplicações inteligentes: Facilita o desenvolvimento de aplicações que utilizam dados semânticos para oferecer funcionalidades avançadas, como recomendações personalizadas, análise semântica e processamento de linguagem natural Allemang and Hendler [2011].

- Pesquisa e análise avançada: Oferece ferramentas poderosas para que pesquisadores e cientistas de dados explorarem redes de dados semânticos complexas, o que permite fazer descobertas e insights que seriam difíceis com bases de dados tradicionais.

A capacidade do GraphDB de lidar com grandes volumes de dados interconectados e a sua habilidade em realizar inferências lógicas tornam-no numa escolha excelente para projetos que dependem intensamente de ontologias complexas e precisam de uma gestão eficiente e dinâmica do conhecimento semântico.

2.4.4 Triplestores

Triplestores, ou armazenamentos de triplos, são bases de dados projetadas especificamente para armazenar e consultar triplos através do uso de SPARQL. Cada triplo num Triplestore consiste num sujeito,

um predicado e um objeto, formatando os dados de acordo com o modelo RDF. Este tipo de armazenamento é ideal para gerir dados semânticos e é fundamental para aplicações que utilizam tecnologias da Web Semântica e ontologias Harris [2013].

Principais Características Os Triplestores são otimizados para armazenar dados altamente interconectados e são capazes de realizar inferências lógicas que ampliam as possibilidades de pesquisa e recuperação de dados Auer and Bizer [2007]. Algumas características notáveis incluem:

- Otimização para dados semânticos: Projetados para armazenar triplos RDF, facilitam a modelação, armazenamento e consulta de dados semânticos.
- Suporte a Consultas Complexas: Suportam a linguagem de consulta SPARQL, permitindo a execução de consultas complexas que podem envolver múltiplos *joins* e operações sobre grafos.
- Inferência e Raciocínio: Muitos triplestores oferecem capacidades de raciocínio que permitem deduzir novos triplos a partir dos existentes, utilizando as ontologias para definir regras lógicas Corcho et al. [2003].

Utilidade com ontologias A integração de triplestores com ontologias proporciona uma poderosa plataforma para gestão de conhecimento e dados semânticos:

- Gestão eficiente de conhecimento: As ontologias fornecem a estrutura necessária para organizar os triplos em triplestores, facilitando o armazenamento lógico e a recuperação eficiente de informações complexas McGuinness and van Harmelen [2003].
- Desenvolvimento de aplicações semânticas: Ao utilizar triplestores em conjunto com ontologias, podem-se construir aplicações que compreendem e processam o significado dos dados, como motores de recomendação, assistentes inteligentes e sistemas de pesquisa avançada Antoniou and van Harmelen [2004].
- Integração e interoperabilidade de dados: Os triplestores podem ser usados para integrar dados de diversas fontes mantendo a semântica definida pelas ontologias, facilitando assim a interoperabilidade entre sistemas diferentes Christian Bizer and Berners-Lee [2009].

A combinação de triplestores com ontologias é particularmente valiosa em domínios onde é necessário fazer a gestão de grandes volumes de dados interligados e onde as relações entre os dados são cruciais para o entendimento e a análise, como em ciências da vida, pesquisa biomédica, e integração de dados empresariais.

2.5 Projetos relacionados

Tal como as ferramentas, há diversos projetos que servem de base e contexto de todo um mundo das ontologias informáticas. Pretende-se enumerar alguns deles e contextualizar a sua importância.

2.5.1 Web Semântica

A web semântica é uma iniciativa que pretende tornar a web mais compreensível para máquinas, a fim de melhorar a interação entre os utilizadores e os sistemas que acedem à informação na web. Para isso são usados metadados e ontologias, que ajudam os sistemas a entender o conteúdo da web de forma mais precisa. Alguns dos objetivos da web semântica incluem tornar a busca na web mais precisa e aumentar a interoperabilidade entre os sistemas que acedem à web.

O termo "web semântica" foi criado pelo cientista da computação Tim Berners-Lee, um dos fundadores da web, em 2001. Ele propôs que a web semântica seria a evolução da web atual, na qual os dados seriam estruturados de forma mais precisa e significativa, permitindo que as máquinas pudessem aceder e processar esses dados de maneira mais simples e eficiente. Uma das principais formas de tornar a web mais compreensível para as máquinas é através do uso de metadados e de ontologias Berners-Lee et al. [2001].

As ontologias são um componente importante da web semântica. São esquemas de categorização que ajudam a definir os termos e as relações entre eles num determinado contexto. Por exemplo, uma ontologia pode definir os termos "cão", "gato" e "animal" e estabelecer relações entre eles, como "cão" e "gato" são tipos de "animal". Isto ajuda as máquinas a entender o significado dos termos e a categorizá-los de forma mais precisa Berners-Lee et al. [2001].

A web semântica também utiliza linguagens de marcação, como o RDF e o OWL, para tornar os dados da web mais estruturados e significativos para as máquinas. Essas linguagens permitem que os dados sejam representados de maneira mais precisa e consistente, o que facilita a sua interpretação pelos sistemas Berners-Lee et al. [2001].

Através disto, é possível ver o uso das ontologias de forma mais abrangente. Levanta-se aqui então a necessidade de um sistema como o que se planeia desenvolver para dar um passo em direção à web semântica, facilitando a construção de ficheiros de dados estruturados, bem como a deteção de erros na estruturação desses mesmos ficheiros.

Ontologias Específicas: Wikidata e CLAV Dentro do espectro da Web Semântica, ontologias como Wikidata e CLAV desempenham papéis essenciais em facilitar o acesso e a organização de conhecimento estruturado de maneira acessível e interoperável.

- **Wikidata:** Uma base de dados livre e aberta de conhecimento geral que serve como um dos principais repositórios de dados estruturados na web. A Wikidata permite a vinculação de informações entre todas as edições linguísticas da Wikipédia, além de fornecer dados para outros projetos da Wikimedia. A plataforma usa uma forma de ontologia para categorizar e relacionar os dados, tornando-os acessíveis tanto para humanos quanto para máquinas wik.

- **CLAV (Classificação e Avaliação da Informação Pública):** Uma ontologia específica que visa a classificação e avaliação de informação no contexto da administração pública portuguesa. CLAV facilita a gestão de informação e a interoperabilidade entre diferentes entidades governamentais, oferecendo um modelo semântico para a organização da informação pública cla.

Essas ontologias são exemplos de como a informação pode ser estruturada de forma a maximizar a utilidade e a acessibilidade numa era digital interconectada, reforçando os objetivos da web semântica em tornar a web um espaço mais inteligível e funcional para diferentes aplicações e usuários.

Além disso, estes exemplos de ontologias não são a área de trabalho desta dissertação. Dada a sua dimensão necessitam de recursos como servidores e grandes máquinas para operar sobre elas. O objeto de estudo são ontologias mais pequenas, para ter um funcionamento local.

Ontologia Específica: ETL A ontologia "Eurico Thomaz de Lima" foi desenvolvida com o objetivo principal de representar o conhecimento a partir do espólio de correspondência do artista, que pertence à Universidade do Minho Rego [2022]. Este projeto académico envolveu a transcrição e anotação da correspondência de Eurico Thomaz de Lima, utilizando diretrizes da TEI e do projeto DALF. As transcrições foram realizadas em XML para preservar a integridade dos documentos originais e facilitar a introdução de metadados, contribuindo para a criação de um arquivo digital pesquisável Ambrósio et al. [2013].

Além disso, a ontologia inclui a transcrição de partituras musicais usando a ferramenta **ABC Notation**, assim como a integração de numerosas fotografias e seus metadados relevantes. Esses elementos não apenas servem como representações visuais e audíveis, mas também enriquecem a ontologia, permitindo uma compreensão mais ampla do espólio Ambrósio et al. [2013].

O valor deste projeto reside na preservação do património biobibliográfico, musical e socio-histórico de Eurico Thomaz de Lima. Abrange uma variedade de disciplinas, incluindo informática, ciências musicais

e humanidades, exemplificando um esforço pioneiro de preservação de património cultural e musical por meio de tecnologias digitais avançadas. Este trabalho notável não apenas organiza e disponibiliza informações sobre a vida e obra de Eurico Thomaz de Lima, mas também estabelece um precedente para futuras iniciativas de documentação digital em áreas similares Ambrósio et al. [2013].

Este é o tipo de ontologias alvo de estudo no desenvolvimento deste trabalho.

2.5.2 RegEx

O módulo **re** do Python desempenha um papel crucial na manipulação e processamento de strings por meio de expressões regulares. Expressões regulares são sequências de caracteres que definem um padrão de pesquisa. Este módulo fornece funcionalidades poderosas para a pesquisa, correspondência e manipulação de texto, tornando-se uma ferramenta indispensável para aqueles que lidam com operações complexas de processamento de strings.

O módulo **re** oferece um conjunto diversificado de funcionalidades, incluindo a capacidade de procurar padrões, fazer substituições, dividir strings e realizar operações avançadas de correspondência.

Embora o módulo **re** seja uma ferramenta poderosa, é importante notar que expressões regulares complexas podem tornar-se difíceis de entender e manter. A busca do equilíbrio entre a complexidade do padrão e a legibilidade do código é uma consideração essencial ao utilizar este módulo Python Software Foundation [2024].

2.5.3 Markdown

O módulo markdown do Python oferece uma solução eficiente para a conversão de texto simples em HTML formatado, seguindo as convenções de linguagem de marcação Markdown. Essa ferramenta é amplamente utilizada para facilitar a criação de documentos com formatação mínima, tornando-se uma escolha popular entre desenvolvedores, escritores e utilizadores que desejam gerar conteúdo web de maneira rápida e intuitiva.

A principal funcionalidade do módulo markdown é a conversão de texto em HTML. Ao utilizar uma sintaxe de marcação simplificada, os utilizadores podem aplicar estilos como negrito, itálico, títulos, listas e links sem a necessidade de codificação HTML extensiva. Além disso, o módulo suporta extensões que expandem suas funcionalidades, permitindo a inclusão de tabelas, códigos destacados e muito mais.

A sintaxe Markdown é conhecida por sua simplicidade e legibilidade. Utilizando caracteres especiais como asteriscos ou *underscores*, os utilizadores podem rapidamente formatar o texto de acordo com suas necessidades. A capacidade de criar cabeçalhos, listas e bloco de citações de forma intuitiva torna

o módulo markdown uma escolha popular para a criação de conteúdo web simples e elegante.

Apesar da praticidade oferecida pelo módulo markdown, é importante reconhecer que pode não ser a melhor escolha para todos os cenários. Aplicações que exigem controle fino sobre a estrutura HTML ou formatação mais avançada podem encontrar limitações na simplicidade do Markdown Python-Markdown Project [2024].

2.5.4 pandoc

O Pandoc é uma ferramenta *open source* que desempenha um papel fundamental na conversão de documentos entre diferentes formatos. Embora não seja um módulo nativo do Python, ele é amplamente utilizado em ecossistemas de desenvolvimento para transformar conteúdo de maneira eficiente e flexível. Nesta secção, exploraremos as principais características, sintaxe e aplicações práticas do Pandoc.

O Pandoc é conhecido por suportar uma vasta gama de formatos, incluindo Markdown, HTML, LaTeX, PDF, DOCX e muitos outros. As funcionalidades principais incluem a capacidade de converter documentos de um formato para outro, aplicar estilos, gerar tabelas de conteúdo automaticamente e processar metadados yaml incorporados nos documentos.

A sintaxe do Pandoc é baseada em linha de comando, e os utilizadores podem especificar opções para personalizar a conversão conforme necessário. Exemplos de uso incluem a conversão de um documento Markdown para PDF Pandoc [2024].

```
pandoc input.md -o output.pdf
```

Além disso, o Pandoc permite a combinação de documentos, inclusão de referências bibliográficas e aplicação de modelos personalizados.

O Pandoc oferece amplas opções para personalização. Os utilizadores podem definir configurações específicas para cada formato de saída, utilizar modelos para controlar a aparência dos documentos e até mesmo criar extensões personalizadas para funcionalidades específicas.

Devido à sua versatilidade, o Pandoc é frequentemente integrado em fluxos de trabalho de desenvolvimento e automação. Pode ser utilizado em scripts e pipelines para realizar conversões automáticas, facilitando a geração de documentos em diferentes formatos conforme necessário.

Enquanto o Pandoc é uma ferramenta poderosa, é importante considerar as dependências do sistema e as diferenças entre as versões. Além disso, a curva de aprendizado pode ser íngreme para utilizadores iniciantes, especialmente ao lidar com personalizações avançadas.

O Pandoc destaca-se como uma ferramenta essencial para aqueles que lidam com a necessidade de converter documentos entre diferentes formatos. A quantidade de funcionalidades, capacidade de personalização e ampla aceitação na comunidade de desenvolvimento tornam-no uma escolha valiosa para projetos que exigem flexibilidade na manipulação de documentos multi-formato.

2.5.5 RDFLib

A RDFLib é uma biblioteca *Python* que permite trabalhar com dados RDF e fornece uma série de ferramentas para trabalhar com esses dados de maneira eficiente Team [2023].

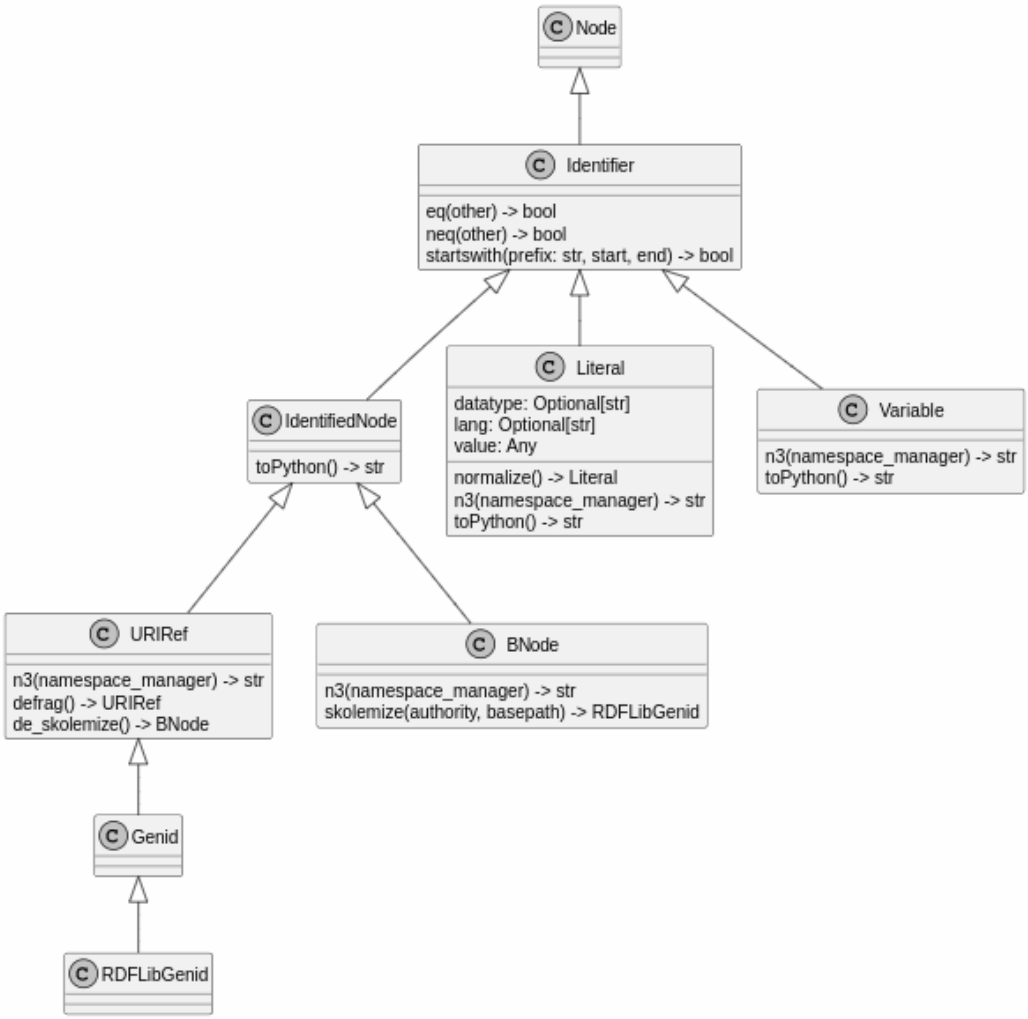


Figura 2: Hierarquia da classe de termos

Funcionalidades

- Carregar e salvar dados RDF em diversos formatos, como XML, Turtle e N-Triples.
- Consultar os dados utilizando SPARQL, uma linguagem de consulta para este tipo de ficheiros.
- Criar, modificar e manipular grafos de maneira fácil.
- Validar os dados de acordo com as especificações da linguagem.

Vantagens

- É uma biblioteca bem documentada e com uma ampla base de utilizadores, o que facilita a obtenção de ajuda e a resolução de problemas Developers [2023].
- Fornece uma série de funcionalidades para trabalhar com dados RDF de maneira eficiente, o que pode ser útil para projetos que envolvam web semântica.
- É uma biblioteca ativamente mantida e atualizada, o que garante a sua compatibilidade com as últimas versões do Python e com as especificações mais recentes.

2.5.6 Owlready2

OWLReady2 é uma biblioteca *Python* que permite trabalhar com ontologias OWL de maneira fácil Lamy [2023]. OWL é uma linguagem de marcação utilizada para representar ontologias, ou seja, esquemas de categorização de termos e relações entre eles. A biblioteca OWLReady2 fornece uma série de ferramentas para trabalhar com essas ontologias de maneira eficiente.

Funcionalidades

- Carregar e salvar ontologias OWL em diversos formatos, como OWL/XML e Turtle.
- Consultar as ontologias utilizando o SPARQL.
- Criar, modificar e manipular ontologias de maneira fácil.
- Validar as ontologias de acordo com as especificações da linguagem.

Vantagens

- É uma biblioteca fácil de usar, com uma sintaxe simples e intuitiva.
- Fornece uma série de funcionalidades para trabalhar com ontologias de maneira eficiente, o que pode ser útil para projetos que envolvam web semântica.
- É uma biblioteca ativamente mantida e atualizada, o que garante a sua compatibilidade com as últimas versões do *Python* e com as especificações mais recentes.

2.5.7 Jinja2: Linguagem de templates

Jinja2 é uma linguagem de *templates* para Python, amplamente utilizada para gerar conteúdo dinâmico aplicações web Projects [2023]. É um dos motores de *template* mais poderosos e flexíveis, suportado por uma sintaxe inspirada na do Django, mas projetada para oferecer uma performance superior e uma maior flexibilidade.

Características principais Jinja2 é conhecido pela sua capacidade de separar a lógica de apresentação do código de aplicação, facilitando a manutenção e a gestão do desenvolvimento de software. As principais características do Jinja2 incluem:

- **Herança de templates:** Jinja2 suporta herança de *templates*, permitindo aos programadores criar um *layout* base que pode ser estendido ou modificado por outros *templates*. Isso promove a reutilização de código e ajuda a manter a consistência do design em toda a aplicação.
- **Blocos e super blocos:** Os programadores podem definir blocos que podem ser substituídos dentro dos *templates* estendidos. Essa funcionalidade é particularmente útil para definir áreas que devem ser personalizadas em cada página, como títulos, cabeçalhos ou rodapés.
- **Filtros e funções personalizadas:** Jinja2 oferece uma série de filtros embutidos para formatação e transformação de dados, como formatação de datas, criação de listas, e aplicação de filtros condicionais. Os programadores também podem definir os eus próprios filtros para usar nos *templates*.

Sintaxe e utilização A sintaxe de Jinja2 é intuitiva e fácil de aprender, especialmente para aqueles familiarizados com outras linguagens de *templates* baseadas em HTML. Um exemplo simples de template Jinja2 pode incluir variáveis, estruturas de controlo e hierarquia:

```
<!DOCTYPE html>
<html lang="pt">
<head>
    <meta charset="UTF-8">
    <title>{{ title }}</title>
</head>
<body>
    {% block content %}
    <h1>Bem-vindo ao meu site, {{ user_name }}!</h1>
    {% endblock %}
</body>
</html>
```

Listing 2.12: Exemplo de template Jinja2

Este exemplo demonstra como as variáveis e os blocos são utilizados em Jinja2 para criar um documento HTML dinâmico.

Aplicações Práticas Jinja2 é bastante utilizado em aplicações web desenvolvidas com Flask, uma micro-framework para Python. No entanto, a sua aplicação estende-se a qualquer contexto em que a geração dinâmica de conteúdo HTML, XML ou qualquer outro formato de texto seja necessária. Isso inclui a geração de relatórios, criação de e-mails dinâmicos e até mesmo geração de código.

Ao permitir uma separação clara entre a lógica de apresentação e a lógica de negócios, Jinja2 melhora significativamente a manutenção e a escalabilidade das aplicações, tornando-se uma escolha popular entre programadores que procuram eficiência e controlo robusto sobre a renderização de interfaces de utilizador.

Parte II

OBOS: Sistema Operativo Orientado às Ontologias

Capítulo 3

Estrutura do OBOS

Este capítulo detalha a arquitetura e a estrutura operacional do OBOS, um Sistema Operativo Orientado às Ontologias desenvolvido para facilitar a gestão e manipulação de ontologias em um ambiente computacional. A necessidade de um sistema operativo dedicado às ontologias surge da crescente complexidade e do volume de dados semânticos gerados em várias áreas, desde a web semântica até sistemas empresariais complexos. O OBOS é projetado para melhorar a "comunicação" com as ontologias, bem como as interações com elas.

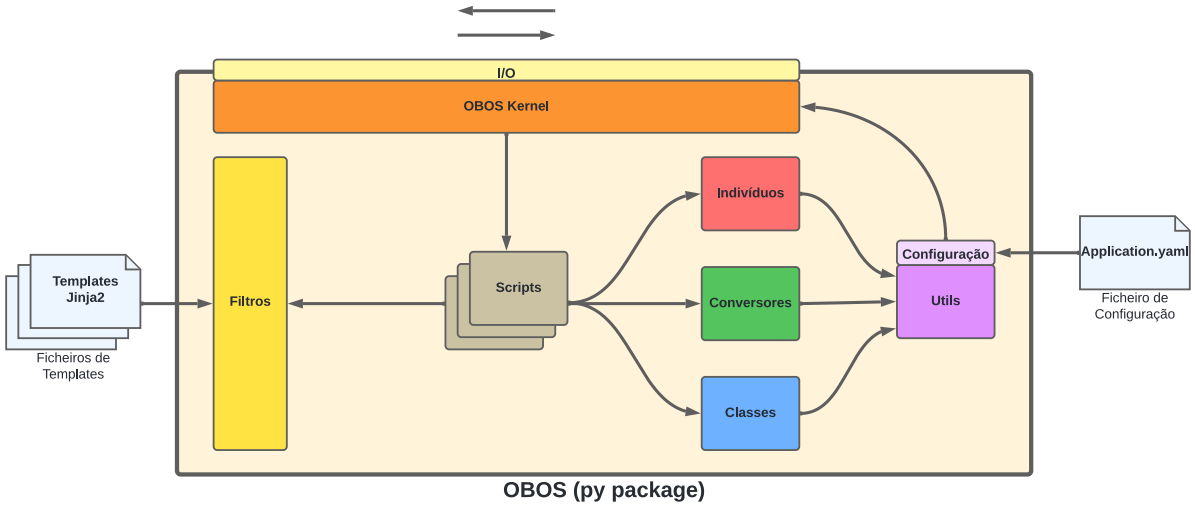


Figura 3: Diagrama da estrutura aplicacional OBOS

3.1 Construção da aplicação

O OBOS é composto por vários componentes que, interligados, são responsáveis pelo processamento dos pedidos do utilizador. A construção desta aplicação teve por base a filosofia UNIX, onde as cada componente tem implementada uma responsabilidade simples e pequena, onde podemos interligar algo maior e criar algo maior, utilizando por exemplo o encadeamento de comandos via *pipe*. No caso concreto

do OBOS, cada componente atua apenas numa área específica do domínio das ontologias e tem diversas funções, todas elas com uma tarefa e implementação simples.

No diagrama 3 podemos observar que quando o utilizador invoca um filtro, passando como argumento a(s) ontologia(s) que forem necessárias para a operação, todo o processamento da(s) mesma(s) passa pelo Kernel.

Posteriormente é excutado um dos Comandos OBOS, representados no diagrama por *scripts*. Esses scripts têm a sua operação própria, invocando um dos outros componentes: Filtros OBOS, indivíduos, conversores ou classes. Todos estes módulos respondem aos scripts. No caso dos filtros, estes dependem de ficheiros de *templates* em formato Jinja2: Linguagem de templates; alguns desses ficheiros já se encontram definidos à partida, de modo a simplificar a interação com o utilizador.

No caso dos **indivíduos** e **classes**, estes contém apenas operações relacionadas com os indivíduos e com as classes da ontologia, respetivamente. Os **conversores** são utilizados para converter as componentes da ontologia em algo mais percetível para o utilizador, que não seja texto.

A componente de **utils** é uma compomente versátil, responsável por operações transversais às ontologias, como interpretar o que o utilizador está a solicitar quando utiliza linguagem natural ou simples operações de como saber se o elemento se trata de um literal ou termo RDF. Tem também funções de limpeza dinâmica de estruturas de dados e agrupamento de dados. Ainda nesta componente, existe uma subcomponente responsável por ler um ficheiro de configurações (*Application.yaml*) e disponibilizar operações de leitura e acesso ao mesmo.

3.1.1 Kernel

O kernel procura ser a parte central do OBOS. Semelhante a um sistema operativo, é responsável pela gestão e armazenamento dos dados da aplicação, criação de cache para aumentar a eficiência, e ainda por gerir todo o *input* e *output*.

É também o kernel que faz a conversão dos ficheiros textuais de ontologia, para objetos e uma representação interna dos dados. Tem assim uma dependência direta do RDFLib. Optou-se por usar esta biblioteca do python uma vez que segue os formatos de ontologias que estamos a trabalhar ao longo deste trabalho: estruturadas, com hierarquia de classes.

Mais detalhes sobre o kernel, podem ser encontrados no capítulo 4.

3.1.2 Scripts

Há vários ficheiros de scripts, um para cada Comandos OBOS. Na verdade, quando algum comando é executado, quem invoca o kernel do sistema operativo é próprio comando, executando sempre a chamada ao kernel para construir a Estrutura Árvore. Só depois disso é que executa a lógica do comando em si.

Os scripts estão divididos desta forma de modo a colocar a documentação no próprio ficheiro, tornando fácil a escrita de documentação. Ainda nos ficheiros de scripts estão disponíveis funções que só sejam utilizadas para a execução do script específico.

3.1.3 Classes e indivíduos

As ontologias são compostas essencialmente por classes, indivíduos e propriedades. Sendo que as propriedades refletem-se nas relações, que está implícito tanto nas classes como nos indivíduos, não foi relevante criar uma componente de propriedades. No entanto estas aparecem nas outras duas componentes.

Na componente de classes estão contidas todas as operações sobre as classes das ontologias, como procuras e *parse* das classes.

Já na componente de indivíduos estão contidas as operações relativas aos indivíduos das ontologias, nomeadamente operações sobre literais, que são sempre mais complexas que as operações sobre termos.

3.1.4 Filtros

Além de toda a integração com os ficheiros de *templates*, é nesta componente que estão definidos todos os filtros descriminados na secção 5.1.5.

Há alguns templates previamente definidos, para ajudar no relação utilizador-máquina, nomeadamente:

- `index.html` → responsável pela criação da página HTML.
- `printDict` → responsável pela apresentação da informação do OBOSVim e OBOSGrep.
- `properties.html` → responsável pela criação da secção de propriedades do HTML geral.
- `section.html` → responsável pela criação de cada um dos blocos do HTML, quer seja de classes, quer seja de indivíduos.

- `singleObj.html` → responsável pela criação do HTML fracionado, isto é, uma única página HTML para cada bloco, em vez de uma única página
- `style.css` → ficheiro CSS responsável pela apresentação estilizada dos conteúdos HTML.

3.1.5 Conversores

O componente de conversores está a ser usado para converter as representações internas dos dados para algo mais percetível pelo utilizador. Por exemplo, quando uma ontologia tem um ficheiro multimédia representado, com a respetiva notação e se esse ficheiro existe na pasta "MEDIA" anexa à ontologia, é possível converter uma simples linha com o nome do ficheiro, na apresentação do próprio ficheiro, como acontece no caso do 5.1.4. Nesse caso, é possível visualizar/ouvir o próprio ficheiro multimédia, em vez de ver apenas o nome do ficheiro.

Outro tipo de conversores implementados, são os conversores textuais de textos formatados, como YAML ou Markdown. Existem ainda os conversores de literais, como datas, que formata-se através da fórmula mais simpática para o utilizador (por defeito por extenso).

3.1.6 Utils

A componente de utils agrupa um conjunto de funções transversais a todo o Sistema Operativo, nomeadamente na linguagem homem-máquina. O OBOS tem de saber que quando o utilizador escreve "pessoa" se refere ao termo `http://obos#pessoa` e não a outro qualquer noutro Namespace.

Para além deste tipo de interligações, é também responsável pela construção de estruturas de dados genéricas para a movimentação dos dados dentro da aplicação. É também através deste componente que são lançados os erros, quer sejam de execução, quer sejam fatais.

Configurações

De modo a tornar a configuração da aplicação mais simples e fácil, foi criado um ficheiro chamado `Application.yaml` que guarda todas as configurações da aplicação. Deste modo, o utilizador pode fazer as alterações que achar necessárias, e modificar sempre que assim se justificar.

```
1 predicates:
2     default:
3         inverso: OWL.inverseOf
4   clean:
5         owl:
```

```
6             inverseOf: inverso de
7 formatos:
8     ttl: n3
9     json: json-ld
10 dir:
11     templates: templates
12 editor: vim
13 conversores:
14     default: string2html
15     md: md2html
16 prioridades:
17     "98": RDFS.subClassOf
18     "0": RDF.type
19     "-1": :see_doc
```

Listing 3.1: Excerto do ficheiro de configurações

No excerto acima, estão expressos alguns campos das configurações, pela seguinte ordem.

1. Definir a que predicado corresponde a língua natural (inverso → `OWL:inverseOf`.
2. Definir a que língua natural corresponde cada predicado (`OWL:inverseOf` → inverso.
3. Definir a que *parser* (RDF) vai ser usado nas extensões dos ficheiros.
4. Definir qual a diretoria de templates padrão da aplicação.
5. Definir o editor padrão da aplicação. Neste caso o vim, mas pode ser o nano, sublime, vscode, entre outros...
6. Definir os conversores a usar para cada formato (textual ou multimédia).
7. Definir a prioridade com as relações aparecem na apresentação da informação. A prioridade é dada por ordem decrescente, ou seja, o de maior número tem prioridade sobre o de menor número. Quando for negativo, segue a mesma lógica, mas definindo os elementos com menos prioridade. Caso seja zero, está-se a definir que não queremos que o elemento seja apresentado. Outros casos não mencionados, serão organizados alfabeticamente entre os elementos com mais prioridade e os elementos com menos prioridade.

Para lidar com este ficheiro de configurações foi construído uma forma de acesso intuitivo para o código. Por exemplo, para aceder ao elemento na linha 3, utiliza-se a seguinte forma:

```
predicates.default.inverso
```

Listing 3.2: Acesso ao ficheiro de configurações

Toda esta lógica, está definida no componente de configurações.

3.2 Tabela de comandos

Comando	Categoria	Descrição
obosclasses	tradução	Devolve as classes de uma ontologia
obossignatures	tradução	Devolve as propriedades de uma ontologia
oboshtml	tradução	Devolve uma representação HTML de uma ontologia
obostemplate	tradução	Devolve a renderização de um *template* com os dados da ontologia
obosgrep	pesquisa	Faz procuras na ontologia, devolvendo apenas os padrões que deram *match*
oboswc [1]	pesquisa	Apresenta algumas estatísticas da ontologia (número de classes, subclasses, indivíduos, ...)
obosquery [1]	pesquisa	Permite efetuar pesquisas na ontologia via SPARQL
obosvisual [1]	pesquisa	Gera gráficos, árvores e redes sobre as informação da ontologia
obosdata [1]	pesquisa	Permite exportar a ontologia em vários formatos
oboscat	edição	Permite juntar várias ontologias numa só
obosvim	edição	Permite editar partes específicas de uma ontologia
oboslint [1]	validação	Permite validar a consistência da ontologia e detetar erros

Tabela 1: Tabela de comandos OBOS

[1] Comando em desenvolvimento

Capítulo 4

Kernel

O foco deste capítulo é o kernel do OBOS, onde as funcionalidades e mecanismos de operação são explorados em profundidade. Será abordado como o kernel faz a gestão dados e processos dentro do SO, destacando a eficiência e a robustez do design. A discussão inclui detalhes sobre a implementação do kernel, os algoritmos utilizados e como estes contribuem para a estabilidade e performance do sistema.

O kernel é parte central de qualquer SO. É o responsável pelo, no arranque do computador, detetar o hardware do computado e arrancar o próprio SO. Após o arranque, fica assim responsável pela gestão de memória, tarefas, arquivos e periféricos.

À semelhança do kernel de um SO vulgar, o kernel do OBOS é o cérebro de toda a aplicação. Desta forma, contém todo o processamento inicial e *parse* dos ficheiros, bem como todas as funcionalidades básicas e transversais a todos os filtros utilizados.

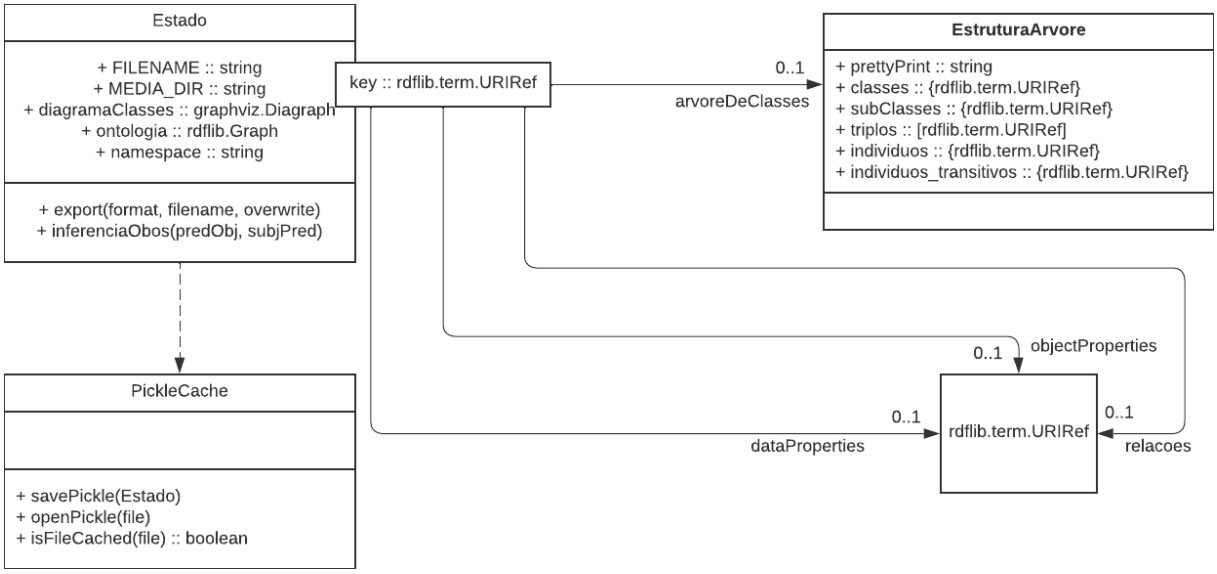


Figura 4: Diagrama de classes do kernel do SO

4.1 Armazenamento de Dados

Esta secção aborda o armazenamento de dados, uma componente crítica de qualquer sistema de informação, focando especialmente nas tecnologias e estratégias utilizadas para gerir e preservar os dados. O armazenamento de dados não se limita apenas à retenção de informações; envolve a organização e acessibilidade para garantir sua integridade e disponibilidade ao longo do tempo. Procura ainda que isso seja feito de forma eficiente.

Ao longo desta secção, procuro fornecer uma explicação detalhada de como os dados são armazenados e geridos.

4.1.1 Armazenamento semi-permanente

Quando são utilizados filtros, por norma o cliente usa repetidamente para a mesma entrada de dados. De modo a aumentar a eficiência e a rapidez do OBOS, esperando este comportamento por parte do utilizador, o sistema aquando a primeira execução de qualquer filtro guarda um *pickle* dos dados armazenados no kernel do SO. Assim, não necessita de fazer todos os cálculos efetuados da primeira vez que executou.

O armazenamento e gestão deste *pickle* é feito tendo em atenção a data de modificação do ficheiro. Tem-se em como exemplo dois ficheiros, uma ontologia e um *pickle*. O carregamento é feito do seguinte modo:

- A data de criação da ontologia é mais recente que a data de criação do *pickle* $\rightarrow$ é criado um novo *pickle*
- A data de criação do *pickle* é mais recente que a data de criação da ontologia $\rightarrow$ é carregado o *pickle*
- A data de modificação da ontologia é mais recente que a data de criação do *pickle* $\rightarrow$ é criado um novo *pickle*
- A data de criação do *pickle* é mais recente que a data de modificação da ontologia $\rightarrow$ é carregado o *pickle*

No caso de não existir um *pickle* associado à ontologia carregada, então é criado um novo *pickle*.

4.1.2 Armazenamento interno

Para armazenar os dados e os cálculos feitos pela aplicação sobre o ficheiro de ontologia, foram criadas duas estruturas de dados (classes).

Estrutura Árvore

A *Estrutura Árvore* é uma estrutura interna de dados que permite criar uma árvore com todo o conteúdo da(s) ontologia(s) carregada(s). Esta estrutura é utilizada para construir a árvore, que é um dicionário de chave-valor, onde o nome da classe é a chave, e o valor é esta mesma estrutura.

Esta estrutura tem a seguinte composição:

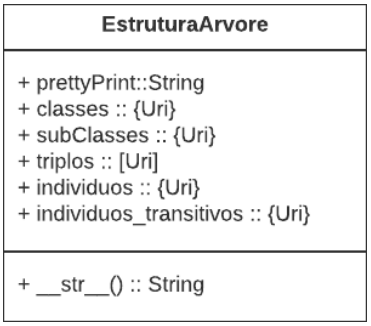


Figura 5: Diagrama da Estrutura Árvore

Basicamente a estrutura árvore serve apenas para guardar dados, não há grandes operações que sejam feitas nesta estrutura. Os campos são:

- **prettyPrint:** A forma preferencial de se escrever a o nome da classe em questão
- **classes:** Um set de todas as classes superiores à classe atual, ou seja, as superclasses
- **subClasses:** um set de todas as classes inferiores à classe atual, ou seja, as subclasses
- **triplos:** todos os triplos onde esta classe é referenciada
- **individuos:** todos os indivíduos da classe atual
- **individuos_transitivos:** todos os indivíduos da classe atual, juntamente com os indivíduos das suas subclasses.

Estado

O estado é a principal estruturo do kernel do OBOS. É o responsável por todos os dados e operações que o kernel pode fazer. Todas as operações passam pela classe de Estado.

Esta classe guarda dados gerais como é o caso do nome do ficheiro, a pasta média (ficheiros auxiliares à ontologia como imagens ou áudios), o namespace usado (se houver algum por defeito) e ainda algumas estruras de dados que são calculadas para facilitar o processamento das ontologias em certas e determinadas operações.

O estado contém dois métodos principais e cruciais para o funcionamento deste SO, como é o caso do export - responsável por guardar a ontologia em vários formatos de ficheiro diferentes e pretendidos e o inferenciaObos - responsável por acrescentar à ontologia triplos que possam ser inferidos e não estejam expressamente declarados.

Além destes métodos, o Estado contém todos os métodos para gerir o Graph do rdflib, ou a estrutura árvore previamente definida. Contém ainda um resumo de todas as relações existentes (predicados) e ainda todas as data e object properties.

Os restantes métodos não explorados nesta secção são métodos auxiliares, que servem como meio para os filtros interagirem com o kernel de uma forma controlada.

4.2 Funcionamento Base

O kernel é responsável por toda as operações bases do SO. Desta forma há uma interligação com todos os filtros utilizados.

Numa primeira instância, o kernel carrega a ontologia para o estado, fazendo todas as operações e estado sobre ela, criando o Estado e a Árvore de Classes.

Numa segunda fase, é responsável por partilhar os resultados com o filtros, para que este tenha conteúdo para fazer as operações. No entanto, todas as operações à ontologia (consultas ou escrita) passam pelo kernel. Só este é que tem controlo sobre as estruturas de dados definidas.

Numa terceira fase (opcional) o kernel exporta a ontologia para um ficheiro no formato solicitado (por defeito o mesmo de entrada). É também o responsável por criar os pickles da ontologia, guardando tudo o que foi criado e calculado na primeira fase.

4.2.1 Parse das classes

Para usufruto de todos os filtros do sistema, é criado e gerido uma estrutura de dados que contém todos os dados essenciais da ontologia. Dentro desta estrutura temos um dicionário (K, V) que mapeia cada classe numa Estrutura Árvore (ver 4.1.2).

Esta estrutura é criada seguindo o seguinte algoritmo:

1. Procurar todas as classes declaradas
    - Todas as classes declaradas são subclasses de OWL:Thing
    - Todas as classes declaradas têm como superclasse OWL:Thing
2. Procurar todos os triplos do tipo (subj, RDF:Type, obj), guardando o obj segundo as regras do ponto 1
3. Procurar todos os triplos do tipo (subj, RDFS:Range, obj), guardando o obj segundo as regras do ponto 1
4. Procurar todas as classes que foram declaradas como subclasses de outras (subj, RDFS:subClassOf, obj)
    - Se o subj já estiver na árvore, então obj é superclasse de subj e subj é subclasse de obj
    - Caso contrário seguir as regras do ponto 1
    - Se o obj já estiver na árvore, então subj é superclasse de obj e obj é subclasse de subj
    - Caso contrário seguir as regras do ponto 1
5. Remover as subclasses de OWL:Thing que estão a mais
    - Se OWL:Thing está no conjunto de superclasses da classe em questão e se esse conjunto tem mais de 1 elemento, então remover classe das subclasses de OWL:Thing e remover OWL:Thing das superclasses da classe em questão

Capítulo 5

Comandos OBOS

Este capítulo apresenta uma visão abrangente dos comandos específicos desenvolvidos para o OBOS. Será discutido como esses comandos interagem com as ontologias, facilitando operações como edição, consulta e manipulação de dados ontológicos. Cada comando será descrito em detalhe, explicando as suas funcionalidades, casos de uso e impacto no funcionamento geral do sistema operativo.

O OBOS é composto por diversos filtros, seguindo a filosofia unix. [2.2]

Os filtros estão definidos um a um, em ficheiros separados, e começam por uma string de documentação, que é usada para o comando –help, transversal a todos os filtros.

Cada um destes filtros tem um funcionamento inicial semelhante, utilizando o jjcli como gestor de opções de entrada. Após essa configuração, o kernel assume o seu papel e faz os cálculos iniciais e carrega os ficheiros de ontologia em memória.

Após este preâmbulo, cada filtro tem o seu próprio comportamento.

5.1 Tradução

As ferramentas de tradução do OBOS são projetadas para oferecer o máximo de flexibilidade e acessibilidade ao interagir com ontologias, facilitando a extração de elementos específicos ou a conversão das ontologias em formatos mais acessíveis e amigáveis. Esta secção detalha vários módulos desenvolvidos ou por desenvolver com o objetivo de simplificar a apresentação de ontologias complexas.

5.1.1 Módulos implementados

O sistema OBOS está equipado com uma variedade de módulos de tradução sofisticados, cada um projetado para facilitar a interação com ontologias por meio da transformação de dados complexos em formatos mais acessíveis e compreensíveis. Esta secção oferece um panorama dos módulos de tradução disponíveis no OBOS, mostrando como cada um contribui para a simplificação da apresentação e ma-

nipulação de dados ontológicos. Os módulos de tradução do OBOS desempenham um papel crucial na adaptação de ontologias para diferentes contextos de uso, permitindo que informações detalhadas sejam convertidas em representações que são tanto práticas quanto alinhadas com as necessidades específicas dos utilizadores. Ao proporcionar ferramentas que convertem, formatam e reestruturam dados ontológicos, esses módulos aumentam significativamente a usabilidade e a acessibilidade das informações, facilitando processos como análise, partilha e integração de dados.

OBOSClasses

O módulo *OBOSClasses* fornece uma visão clara e estruturada da taxonomia das classes dentro da ontologia. É fundamental para qualquer utilizador que necessite compreender a hierarquia e a estrutura organizacional dos dados ontológicos. Este módulo facilita significativamente a navegação e a compreensão das relações entre diferentes classes, promovendo uma melhor interpretação e utilização dos dados.

OBOSSignatures

Segue-se o módulo *OBOSSignatures*, que exibe as assinaturas de cada classe na ontologia. As assinaturas incluem propriedades e tipos de dados que definem cada classe, proporcionando *insights* essenciais sobre os atributos que cada elemento pode possuir. Este módulo é crucial para os desenvolvedores e pesquisadores que estão interessados em validar dados ou desenvolver aplicações que dependem da precisão e da integridade da informação ontológica.

OBOSHtml

O módulo *OBOSHtml* transforma informações ontológicas complexas em páginas HTML interativas e visualmente atraentes. Esta conversão não só torna os dados mais acessíveis a um público mais amplo, incluindo aqueles sem especialização em ontologias, mas também facilita a apresentação e a educação, permitindo uma exploração visual dos dados.

OBOSTemplate

Por fim, *OBOSTemplate* oferece a capacidade de gerar qualquer tipo de documento a partir de templates Jinja2, personalizando a saída de dados para atender a necessidades específicas. Este módulo é extremamente versátil, suportando a criação de uma variedade de formatos de documentos, desde relatórios até arquivos de dados estruturados, integrando-se facilmente com outras ferramentas e sistemas.

5.1.2 OBOSClasses

O filtro das classes, *OBOSClasses*, permite ao utilizador gerar uma taxonomia das classes, proporcionando uma visão clara e organizada das classes e suas respectivas subclasses. Além da representação hierárquica, o filtro também indica o número de indivíduos associados a cada classe, contabilizando tanto os membros diretos quanto os transitivos. Por exemplo, se a classe 'Engenheiro' não tem indivíduos diretamente associados mas possui uma subclasse 'Engenheiro Informático' com um indivíduo, o resultado exibido será:

```
1 Engenheiro (0,1)
2     Engenheiro Informático (1,1)
```

Listing 5.1: Exemplo de output

Linha 1: O '0' na classe 'Engenheiro' indica que não há indivíduos diretamente associados, enquanto o '1' revela a presença de um indivíduo transitivo proveniente de 'Engenheiro Informático'.

Linha 2: O '1' na classe 'Engenheiro Informático' representa um indivíduo direto, e o segundo '1' indica a mesma contagem considerando a transitividade, ou seja, o mesmo indivíduo.

Para aumentar a versatilidade do filtro, foram definidas algumas *flags* que permitem a geração de outputs diferenciados conforme as necessidades do utilizador:

- **-t** → Permite definir um topo específico para a taxonomia, focando a análise em uma ou mais classes selecionadas.
- **-l** → Limita o número de níveis de subclasses a serem exibidos, facilitando a visualização em ontologias muito densas.
- **-g** → Gera uma representação gráfica da taxonomia, criando um diagrama que ilustra as relações entre as classes de forma visual.

Este filtro é fundamental para a análise e compreensão das estruturas ontológicas, especialmente em contextos onde a precisão e a clareza são cruciais. Se uma classe está sendo usada como tal mas não foi explicitamente declarada, ela será identificada e incluída na taxonomia, assegurando que nenhuma informação relevante seja omitida. Além disso, todas as classes são consideradas subclasses de um topo universal, normalmente 'OWL:Thing' no contexto do OBOS, garantindo que a taxonomia seja completa e abrangente.

Necessidade

A necessidade do filtro *OBOSClasses* manifesta-se claramente quando se considera a complexidade e a vastidão das ontologias modernas. Em ambientes académicos e de pesquisa, onde a manipulação precisa de dados ontológicos é frequentemente necessária, entender a hierarquia e as relações entre as classes torna-se indispensável. Este filtro facilita não apenas a visualização e a compreensão da estrutura ontológica, mas também ajuda na identificação de erros e na validação de dados, contribuindo significativamente para a integridade e a utilidade das informações ontológicas.

5.1.3 OBOSSignatures

O filtro *OBOSSignatures* é uma ferramenta poderosa no sistema OBOS, projetada para extrair e catalogar as propriedades de objetos (ObjectProperties) e propriedades de dados (DataProperties) de uma ontologia. Este filtro é essencial para os utilizadores que necessitam de uma compreensão clara e organizada das várias propriedades que compõem a ontologia, fornecendo uma base para análises mais aprofundadas ou para o desenvolvimento de aplicações que interagem com esses dados.

```
[Object Properties]
authorOf

[Data Properties]
descrição
```

Listing 5.2: Exemplo de output

O *OBOSSignatures* não só lista as propriedades conforme são oficialmente definidas na ontologia, mas também identifica e inclui propriedades que estão sendo utilizadas mas não foram explicitamente declaradas. Isso é possível graças ao algoritmo avançado do filtro que analisa o uso de cada propriedade dentro da ontologia, determinando se uma propriedade deve ser classificada como Object Property ou Data Property baseada no tipo de dados do contradomínio. Por exemplo, se a propriedade 'authorOf' está associada a um indivíduo, ela é categorizada como uma Object Property. Se estiver associada a um literal, é categorizada como uma Data Property.

Esta funcionalidade não só aumenta a utilidade do filtro ao proporcionar uma visão completa das propriedades usadas na ontologia, independentemente de sua declaração formal, mas também ajuda a identificar incoerências ou erros na definição das ontologias. A presença de uma propriedade em ambas as listas de Object e Data Properties pode indicar uma duplicidade de uso que necessita correção, fornecendo uma oportunidade para melhorar a qualidade e a precisão da ontologia.

Essa capacidade de diagnóstico e a flexibilidade na geração de listas em vários formatos (JSON, Turtle, YAML ou texto simples) fazem do *OBOSSignatures* uma ferramenta indispensável para qualquer ontologista ou desenvolvedor que trabalhe com sistemas baseados em ontologias, facilitando tanto a manutenção quanto o desenvolvimento de novas funcionalidades com base nas propriedades identificadas.

Necessidade

O módulo *OBOSSignatures* é fundamental para o sistema OBOS, oferecendo capacidades essenciais para a análise detalhada e a gestão de ontologias. A necessidade deste módulo deriva da complexidade inerente às ontologias e da importância de entender as propriedades que definem e conectam seus diversos elementos. Ao extrair e catalogar as ObjectProperties e DataProperties, o *OBOSSignatures* permite aos utilizadores obter uma visão clara das capacidades e restrições de cada classe e indivíduo dentro da ontologia.

Clarificação e organização As ontologias podem-se tornar extremamente complexas, com inúmeras classes e propriedades que podem ser difíceis de navegar sem uma ferramenta dedicada. O *OBOSSignatures* simplifica esta complexidade ao apresentar uma lista organizada de propriedades, o que é crucial para desenvolvedores e pesquisadores durante a fase de modelação ou modificação de ontologias. Esta organização ajuda a evitar erros de implementação e a garantir que as inter-relações entre os dados sejam mantidas corretamente.

Verificação da consistência e integridade Além disso, o *OBOSSignatures* desempenha um papel vital na verificação da consistência e integridade das ontologias. Ao identificar propriedades que são utilizadas mas não formalmente declaradas, o módulo auxilia na identificação de possíveis inconsistências ou omissões na definição da ontologia. Esta capacidade é especialmente valiosa em ambientes onde a precisão dos dados é crítica, como em aplicações médicas, jurídicas ou académicas.

Facilitação da interoperabilidade Finalmente, ao fornecer a capacidade de gerar listas de propriedades em vários formatos, como JSON, Turtle, e YAML, o *OBOSSignatures* facilita a interoperabilidade entre diferentes sistemas e plataformas. Isso permite que ontologias sejam mais facilmente integradas em diferentes ambientes de software, promovendo a reutilização de dados e a colaboração entre diferentes equipas e organizações.

Em resumo, o *OBOSSignatures* é indispensável para qualquer utilizador que necessite de uma compreensão profunda e de gestão eficaz das propriedades de uma ontologia, garantindo que as informações

sejam utilizadas de forma coerente e eficiente.

5.1.4 OBOSHtml

O filtro *OBOSHtml* é uma das ferramentas mais visuais do sistema OBOS, permitindo aos utilizadores gerar uma página HTML estática que representa de forma abrangente os elementos da ontologia e suas inter-relações. Este filtro é projetado para fornecer uma visão clara e interativa da ontologia, facilitando tanto a educação quanto o desenvolvimento e a análise de ontologias.

Estrutura da página HTML A estrutura da página gerada pelo *OBOSHtml* é meticulosamente organizada para promover a compreensão e a navegação eficiente. Inicialmente, a página apresenta uma taxonomia de classes que é exportada conforme descrito na secção 5.1.2. Esta representação hierárquica das classes proporciona uma visão geral rápida da estrutura ontológica e é seguida por uma listagem detalhada das classes, que são organizadas alfabeticamente.

Após as classes, os indivíduos são apresentados, também organizados alfabeticamente. Cada indivíduo é detalhado com links para suas propriedades e relações pertinentes, oferecendo um meio prático de explorar as entidades da ontologia. Finalmente, as propriedades são divididas em Data Properties e Object Properties, cada uma listada separadamente para clareza e fácil acesso.

Design visual e código de cores Um aspeto distintivo do *OBOSHtml* é seu uso de um código de cores específico, que é cuidadosamente aplicado para diferenciar entre classes, indivíduos e propriedades. Este sistema de cores não apenas enriquece a interface visual, mas também auxilia os utilizadores na rápida identificação dos diferentes tipos de elementos dentro da ontologia. Por exemplo, as classes são destacadas em azul, os indivíduos em vermelho e as propriedades em verde, como mostrado nas figuras 6, 7, e 8.

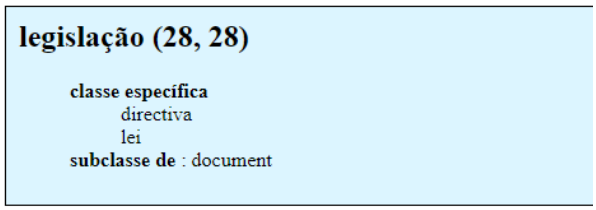


Figura 6: Exemplo de uma Classe - Azul

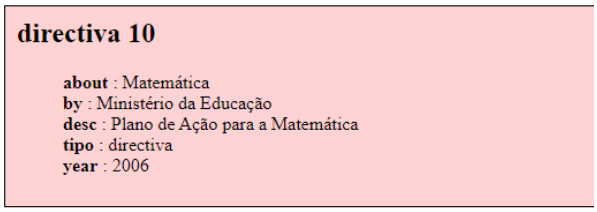


Figura 7: Exemplo de um Indivíduo - Vermelho

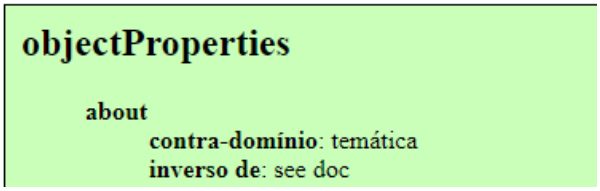


Figura 8: Exemplo de uma Propertie - Verde

Funcionalidades adicionais Além de sua estrutura principal e design visual, o *OBOSHtml* incorpora várias funcionalidades que aumentam sua utilidade. Por exemplo, os utilizadores podem interagir com a página HTML através de elementos interativos, como botões ou links que expandem ou colapsam as descrições das classes e indivíduos, facilitando a navegação em ontologias particularmente densas ou complexas.

Este filtro, portanto, não apenas visualiza a ontologia de maneira eficaz mas também serve como uma ferramenta educacional e de desenvolvimento, ajudando os utilizadores a entender e trabalhar com ontologias de maneira mais intuitiva e produtiva.

5.1.5 OBOSTemplate

O script *OBOSTemplate* é uma ferramenta avançada dentro do sistema OBOS que permite aos utilizadores gerar documentos personalizados a partir de templates Jinja2, utilizando dados extraídos diretamente da ontologia. Este script é essencial para transformar dados ontológicos complexos em documentos legíveis e formatados, adequados para apresentações, relatórios técnicos, ou até mesmo para geração de conteúdo web dinâmico.

Funcionamento básico O *OBOSTemplate* opera combinando a flexibilidade do motor de templates Jinja2 com a robustez dos dados ontológicos. Jinja2 é uma linguagem de templates popularmente utilizada para criar arquivos de texto formatados, como HTML, LaTeX, ou documentos simples. O script extrai dados da ontologia, como classes, propriedades, e relações, e os insere em pontos específicos dentro de um template Jinja2 pré-definido pelo utilizador.

Processo de integração de dados Ao executar *OBOSTemplate*, o utilizador precisa especificar o caminho para o arquivo de ontologia e o template Jinja2 que deseja usar. O script então analisa a ontologia, extrai os dados necessários e passa-os para o template. Durante esse processo, o script pode executar várias funções de filtragem e ordenação para garantir que os dados inseridos no documento final estejam corretos e organizados conforme a necessidade do utilizador.

Exemplo de uso Um exemplo prático do uso do *OBOSTemplate* pode ser a geração de um relatório sobre uma ontologia de dispositivos médicos, onde cada classe (como diferentes tipos de dispositivos) e suas respetivas propriedades (como especificações ou normas aplicáveis) precisam ser listadas de forma organizada. O utilizador cria um template Jinja2 que define como e onde cada tipo de dado deve aparecer no documento final, e o *OBOSTemplate* preenche esse template com dados reais extraídos da ontologia.

Opções avançadas O script também inclui várias opções avançadas que aumentam sua utilidade:

- **-o**: Direciona a saída do programa para um arquivo específico, permitindo que o utilizador salve o documento gerado diretamente no sistema operativo.
- **-t**: Permite ao utilizador selecionar um template específico, dando flexibilidade para trabalhar com múltiplos formatos e designs de documento.
- **–objects**: Aceita um mapa de objetos, que são variáveis adicionais que podem ser passadas para o template, proporcionando uma maior personalização e controlo sobre o conteúdo gerado.

Importância e aplicabilidade O *OBOSTemplate* é particularmente útil em ambientes onde a apresentação precisa e personalizada de dados ontológicos é crucial. Ele permite aos cientistas de dados, pesquisadores e técnicos apresentar suas descobertas de forma mais eficaz, criando documentos que são ao mesmo tempo informativos e esteticamente agradáveis. Além disso, a capacidade de automatizar a geração de documentos a partir de ontologias pode significar uma economia considerável de tempo e esforço em projetos grandes.

Em suma, *OBOSTemplate* transforma a complexidade dos dados ontológicos em documentos formatados e personalizados, facilitando o entendimento e a comunicação das informações contidas nas ontologias de forma mais eficaz e adaptada às necessidades específicas do utilizador.

Filtros OBOS

O arquivo *filtrosjinja2* é parte integrante do sistema OBOS e fornece uma série de filtros personalizados que são utilizados com o motor de templates Jinja2. Estes filtros facilitam a interação direta com as ontologias, permitindo a manipulação dinâmica dos dados ontológicos diretamente nos templates. Esta secção explora as principais funções definidas no arquivo e suas aplicações.

Funções definidas

1. **j2_rel(subj, pred)**: Retorna objetos que correspondem a um sujeito e predicado especificados. Esta função é útil para extrair informações específicas de uma ontologia, com uma opção para limpeza dos resultados.

2. **j2_relfun(subj, pred, preText = ", posText = ")**: Semelhante à função anterior, mas permite a concatenação de texto antes e depois do resultado. Isso é particularmente útil para formatar a saída diretamente dentro de templates.

3. **j2_relchain(subj, *listPred, preText = ", posText = ", one=True)**: Realiza uma pesquisa de predicados em cadeia a partir de um sujeito inicial, permitindo a navegação através de relações complexas dentro da ontologia. Retorna o primeiro objeto se *one* é verdadeiro, ou uma lista de objetos se falso.

4. **j2_triplosobj(obj)**: Retorna todos os predicados e sujeitos relacionados a um objeto, útil para análise de como certos dados estão interconectados dentro da ontologia.

5. **j2_triplossubj(subj)**: Retorna todos os predicados e objetos relacionados a um sujeito, facilitando a visualização de todas as conexões que emanam de um ponto específico na ontologia.

6. **j2_triplosExept(subj, *listaExcept)**: Funciona como a função anterior, mas exclui relações especificadas por uma lista de predicados, permitindo filtrar dados irrelevantes ou sensíveis.

7. **j2_triplosBeauty(triplo)**: Formata um tuplo de dados ontológicos (triplo) numa string mais legível, melhorando a apresentação visual dos dados.

8. **j2_isLiteral(string : str)**: Verifica se uma string é um literal dentro do contexto da ontologia, auxiliando na validação de tipos de dados.

9. **j2_isUriRef(string : str)**: Verifica se uma string é uma referência de URI, importante para assegurar que as ligações dentro da ontologia sejam válidas.

Cada função é projetada para facilitar operações específicas dentro de templates Jinja2, melhorando significativamente a eficiência e a eficácia da manipulação e apresentação de dados de ontologias no sistema OBOS.

Utilidades e aplicações Estes filtros permitem uma integração eficiente e direta com as ontologias, promovendo:

- **Integração Direta**: Incorporação de consultas ontológicas diretamente nos templates, simplificando o processo de geração de documentos.
- **Personalização**: Suporte à inserção de formatação personalizada, como HTML ou texto, para apresentar os dados de maneira mais clara e esteticamente agradável.
- **Eficiência**: Redução da necessidade de processamento de dados no backend, permitindo uma geração de documentos mais rápida e dinâmica.

Exemplo prático Um exemplo de aplicação desses filtros em um template Jinja2 para um relatório de funcionários poderia ser descrito como segue:

```
{% for funcionario in funcionarios %}
  Nome: {{ funcionario.nome }}
  Departamento: {{ j2_rel(funcionario, 'departamento') }}
  Supervisor: {{ j2_relchain(funcionario, 'departamento', 'supervisor') }}
{% endfor %}
```

Listing 5.3: Exemplo de template

Este código ilustra a aplicação das funções para extrair informações encadeadas sobre funcionários diretamente dentro de um template, demonstrando a flexibilidade e a potência dos filtros personalizados.

Essas características fazem dos filtros aplicacionais uma ferramenta extremamente valiosa para qualquer utilizador que precise manipular e apresentar dados de ontologias de maneira eficiente e personalizada dentro do sistema OBOS.

5.2 Pesquisa

As capacidades de pesquisa do OBOS são essenciais para explorar e analisar eficazmente grandes volumes de dados ontológicos, fornecendo aos utilizadores ferramentas poderosas para descobrir *insights* e conexões dentro de complexas estruturas de informação. Esta secção explora em detalhe as várias

componentes e técnicas de pesquisa incorporadas no sistema, desenhadas para maximizar a eficiência e a precisão na recuperação de informações. Através de uma combinação de pesquisa avançada, funcionalidades de filtragem e interfaces intuitivas, o OBOS permite aos utilizadores não apenas aceder rapidamente os dados desejados, mas também realizar análises profundas e contextualizadas, essenciais para aplicações académicas, científicas e industriais.

5.2.1 Módulos implementados

O sistema OBOS incorpora uma série de módulos de pesquisa avançada, cada um projetado para atender a necessidades específicas de interação e análise de ontologias. Esta seção proporciona um resumo dos principais módulos de pesquisa implementados no OBOS, destacando suas funcionalidades e o impacto de cada um no processo de gestão de conhecimento dentro do sistema.

OBOSGrep

O módulo *OBOSGrep* é uma ferramenta de pesquisa que permite aos utilizadores procurar por elementos específicos dentro das ontologias. Inspirado pelo comando Unix *grep*, este módulo oferece uma maneira poderosa e flexível de filtrar dados através de critérios definidos pelo utilizador, como nomes de classes, propriedades ou qualquer outro termo relevante. A funcionalidade de pesquisa avançada é essencial para utilizadores que precisam rapidamente localizar e analisar partes específicas de uma ontologia complexa.

OBOSWc

O *OBOSWc* adapta o conceito do comando Unix 'wc' (word count) para ontologias, fornecendo uma funcionalidade essencial para contar rapidamente elementos como classes, propriedades e indivíduos dentro de uma ontologia. Esta ferramenta simplifica a análise quantitativa da estrutura ontológica, facilitando a avaliação rápida da complexidade e do tamanho de uma ontologia. É uma ferramenta prática para utilizadores que necessitam de insights imediatos sobre os componentes de suas bases de dados ontológicas.

OBOSQuery

O módulo *OBOSQuery* estende as capacidades de pesquisa do OBOS ao permitir consultas complexas utilizando uma linguagem de consulta estruturada. Este módulo é crucial para realizar análises profundas e extrair relações ou padrões que não são imediatamente aparentes. Ele suporta várias formas de

consultas, incluindo SPARQL, o que o torna extremamente versátil para pesquisa académica e desenvolvimento industrial, onde consultas precisas e eficientes são necessárias para manipular grandes conjuntos de dados ontológicos.

OBOSVisual

O *OBOSVisual* é um módulo dedicado à visualização de dados de ontologia. O objetivo é transformar os dados complexos e muitas vezes abstratos de uma ontologia em representações gráficas claras e compreensíveis, como gráficos, árvores e redes. Esta funcionalidade não apenas facilita a compreensão das estruturas de dados por parte dos utilizadores, mas também ajuda na identificação de conexões e na análise de dependências dentro da ontologia. A visualização é particularmente útil em apresentações académicas e reuniões de equipa, onde uma compreensão clara dos dados é essencial.

OBOSData

Finalmente, o *OBOSData* é um módulo projetado para a gestão e exportação de dados. O objetivo é permitir que os utilizadores exportem dados da ontologia em vários formatos, incluindo XML, JSON e CSV, facilitando a interoperabilidade e o partilha de dados entre diferentes plataformas e sistemas. Este módulo é vital para a preservação de dados, relatórios e para a integração de ontologias em outros sistemas e aplicações.

Cada um destes módulos de pesquisa implementados no OBOS é projetado com o objetivo de maximizar a acessibilidade, eficiência e usabilidade dos dados ontológicos, garantindo que os utilizadores possam não apenas aceder e manipular, mas também partilhar e apresentar suas informações de maneira eficaz.

5.2.2 OBOSGrep

O módulo *OBOSGrep* é uma ferramenta de pesquisa robusta dentro do sistema OBOS, inspirada na funcionalidade do comando Unix ‘grep‘, que é amplamente utilizado para procurar expressões dentro de arquivos de texto. Este módulo adapta essa capacidade para o contexto de ontologias, permitindo aos utilizadores realizar pesquisas detalhadas por termos específicos dentro de estruturas de dados ontológicas complexas.

Funcionalidades do OBOSGrep O *OBOSGrep* permite que os utilizadores filtrem e localizem rapidamente elementos dentro de uma ontologia, como classes, propriedades, indivíduos ou qualquer fragmento

de texto específico contido nos dados. A ferramenta é particularmente útil para:

- Identificar e extrair partes específicas da ontologia que correspondem a critérios de pesquisa definidos pelo utilizador.
- Realizar análises rápidas de consistência ou completude da ontologia.
- Auxiliar no *debug* e no desenvolvimento de ontologias, fornecendo uma maneira rápida de verificar a presença ou ausência de determinados dados.

Como funciona o OBOSGrep O funcionamento do *OBOSGrep* baseia-se na leitura dinâmica da ontologia carregada no sistema. Os utilizadores podem especificar uma string ou expressão regular que desejam procurar, e o módulo verifica toda a base de dados da ontologia para encontrar correspondências. As principais características incluem:

1. **Sensibilidade ao Contexto**: O *OBOSGrep* entende o contexto de ontologias e pode diferenciar entre tipos de elementos, como diferenciar entre uma classe e uma propriedade com nomes similares.
2. **Expressões Regulares**: Suporte a expressões regulares para pesquisas complexas que podem incluir variações de um termo, padrões específicos, ou condições compostas.
3. **Desempenho Otimizado**: Implementação eficiente que minimiza o tempo de resposta, mesmo ao lidar com grandes volumes de dados ontológicos.

Aplicações práticas Na prática, o *OBOSGrep* é utilizado em várias situações dentro do fluxo de trabalho de desenvolvimento e manutenção de ontologias. Por exemplo, um desenvolvedor pode usar o módulo para verificar rapidamente se uma nova classe ou propriedade foi corretamente adicionada à ontologia ou para encontrar todas as instâncias de uma classe específica que precisam ser atualizadas após uma mudança nos requisitos do projeto.

Interface do utilizador O *OBOSGrep* é acedido através de uma interface de linha de comando no sistema OBOS, onde os comandos podem ser inseridos diretamente pelos utilizadores. Isso proporciona uma maneira flexível e poderosa de interagir com dados ontológicos, refletindo a filosofia Unix de ferramentas "pequenas, mas poderosas", que fazem bem uma coisa específica.

O módulo *OBOSGrep* representa uma parte essencial do conjunto de ferramentas de pesquisa do OBOS, oferecendo aos utilizadores capacidades avançadas de pesquisa e recuperação de dados dentro

de ontologias complexas, com a flexibilidade e a eficácia necessárias para suportar uma ampla gama de tarefas de desenvolvimento e manutenção de ontologias.

5.2.3 OBOSWc

O módulo *OBOSWc*, inspirado no comando Unix 'wc' (word count), é adaptado no sistema OBOS para quantificar elementos específicos dentro de ontologias. Esta ferramenta permite aos utilizadores contar elementos como classes, propriedades e indivíduos dentro de uma ontologia, fornecendo uma visão quantitativa rápida e eficaz da estrutura e complexidade dos dados ontológicos.

Funcionalidades do OBOSWc O *OBOSWc* é capaz de:

- Contar o número total de **classes** dentro da ontologia.
- Determinar a quantidade de **propriedades**, incluindo DataProperties e ObjectProperties.
- Totalizar os **indivíduos** ou instâncias presentes na ontologia.

Processo de operação O funcionamento do *OBOSWc* envolve:

1. **Leitura da Ontologia:** O módulo carrega e analisa a estrutura da ontologia armazenada no sistema OBOS.
2. **Contagem dos Elementos:** Executa a contagem baseada nos critérios especificados pelo utilizador.
3. **Relatório dos Resultados:** Os resultados são apresentados em um formato de fácil leitura, disponíveis no terminal ou como um relatório exportável.

Aplicações práticas As aplicações do *OBOSWc* incluem:

- **Auditorias de Ontologia:** Facilita revisões rápidas da estrutura de uma ontologia, útil para auditorias de qualidade ou preparação para atualizações significativas.
- **Desenvolvimento e Manutenção:** Ajuda os desenvolvedores a compreender a escala de uma ontologia, auxiliando no planeamento de modificações ou otimizações.

Este módulo demonstra como o OBOS integra funcionalidades clássicas de sistemas operacionais num contexto de gestão de ontologias, proporcionando ferramentas eficientes para a análise e manipulação de estruturas de dados complexas.

5.2.4 OBOSQuery

O módulo *OBOSQuery* é uma ferramenta avançada de consulta no sistema OBOS que permite aos utilizadores realizar pesquisas complexas e precisas dentro de ontologias. Projetado para interagir com grandes conjuntos de dados ontológicos, este módulo facilita a extração de informações específicas utilizando a linguagem de consulta SPARQL, reconhecida mundialmente por sua capacidade em lidar com dados RDF.

Funcionalidades do OBOSQuery As principais funcionalidades do *OBOSQuery* incluem:

- **Suporte a SPARQL:** Permite a realização de consultas detalhadas e complexas, explorando a profundidade das ontologias.
- **Extração de Dados Complexos:** Capacidade de efetuar consultas que interligam várias classes e propriedades, permitindo análises aprofundadas e a descoberta de relações ocultas.
- **Filtros Avançados:** Oferece opções para refinar resultados através de filtros e funções aplicadas diretamente nas consultas.

Processo de operação O funcionamento do *OBOSQuery* é dividido nas seguintes etapas:

1. **Interface de Consulta:** Os utilizadores inserem consultas SPARQL por meio de uma interface de linha de comando ou gráfica.
2. **Processamento de Consultas:** As consultas são analisadas, sintaticamente verificadas e executadas contra a base de dados da ontologia.
3. **Otimização de Consultas:** O módulo aplica técnicas de otimização para melhorar a eficiência das consultas, como o reordenamento de filtros e pré-carga de dados.
4. **Retorno de Resultados:** Os resultados são formatados e apresentados de acordo com as especificações da consulta, em formatos como tabelas ou gráficos.

Aplicações práticas O *OBOSQuery* tem uma ampla gama de aplicações, incluindo:

- **Pesquisa Académica:** Utilizado por estudantes para extrair informações específicas para análises científicas.

- **Desenvolvimento de Aplicações:** Auxilia desenvolvedores na integração de funcionalidades de busca avançada em aplicações.
- **Análise de Dados em Empresas:** Empregado por empresas para análise estratégica de dados, facilitando a tomada de decisões baseada em informações ontológicas complexas.

Esta ferramenta é vital para o sistema OBOS, trazendo capacidades de consulta avançadas que são essenciais para manipular e analisar efetivamente dados ontológicos em vários contextos profissionais e académicos.

5.2.5 OBOSVisual

O módulo *OBOSVisual* é uma ferramenta poderosa dentro do sistema OBOS, dedicada à visualização gráfica de ontologias. Este módulo transforma dados ontológicos complexos em representações visuais claras e interativas, facilitando a compreensão e análise das estruturas de dados por parte dos utilizadores.

Funcionalidades do OBOSVisual O *OBOSVisual* oferece diversas funcionalidades para a visualização de ontologias:

- **Representação Gráfica de Classes e Relações:** Visualiza classes, subclasses, e suas inter-relações em forma de gráficos ou diagramas hierárquicos.
- **Mapeamento de Indivíduos e Propriedades:** Permite a visualização de indivíduos associados a cada classe e as propriedades que interligam esses elementos.
- **Interatividade:** Os utilizadores podem interagir com as visualizações, como ampliar, reduzir e explorar partes específicas da ontologia para melhor compreensão.

Processo de operação O funcionamento do *OBOSVisual* envolve várias etapas técnicas para transformar dados ontológicos em imagens úteis:

1. **Extração de Dados:** O módulo lê e interpreta a estrutura da ontologia carregada, identificando elementos-chave como classes, propriedades e relações.
2. **Geração de Gráficos:** Utiliza algoritmos de visualização para organizar e desenhar os dados em formatos gráficos, aplicando técnicas de layout gráfico para otimizar a clareza e legibilidade.
3. **Renderização:** As imagens são renderizadas numa interface gráfica, onde elementos visuais são ajustados para maximizar a usabilidade e a experiência do utilizador.

Aplicações práticas As imagens geradas pelo *OBOSVisual* são essenciais em diversas aplicações:

- **Educação e Treino:** Facilita o ensino de conceitos complexos de ontologias em contextos académicos.
- **Análise de Dados:** Suporta analistas e cientistas de dados na exploração de grandes conjuntos de dados ontológicos, identificando padrões e anomalias.
- **Desenvolvimento de Ontologias:** Auxilia desenvolvedores de ontologias no processo de design e refinamento, oferecendo uma perspetiva visual das estruturas que estão sendo criadas ou modificadas.

Em suma, o *OBOSVisual* é uma ferramenta indispensável no OBOS, proporcionando não apenas uma maneira eficaz de visualizar e interagir com dados ontológicos, mas também enriquecendo a capacidade dos utilizadores de compreender e manipular complexidades inerentes a esses conjuntos de dados.

5.2.6 OBOSData

O módulo *OBOSData* no sistema OBOS é projetado para a gestão eficiente e exportação de dados ontológicos, permitindo aos utilizadores converter e guardar esses dados numa variedade de formatos padrão. Esta funcionalidade é crucial para a utilização de ontologias em diferentes ambientes de software e plataformas, garantindo que os dados possam ser facilmente acedidos e utilizados por diversas aplicações e serviços.

Funcionalidades do OBOSData As principais funcionalidades do *OBOSData* incluem:

- **Conversão de Formatos:** Permite a conversão de dados ontológicos para vários formatos, como XML, JSON, CSV e RDF, atendendo às necessidades de interoperabilidade e integração de sistemas.
- **Exportação de Dados:** Os utilizadores podem exportar dados completos ou parciais da ontologia, selecionando especificamente quais partes dos dados são necessárias para suas aplicações.
- **Preservação da Integridade dos Dados:** Garante que todas as conversões e exportações mantenham a integridade estrutural e semântica dos dados ontológicos.

Processo de operação O funcionamento do *OBOSData* é caracterizado pelas seguintes etapas:

1. **Seleção de Dados:** Os utilizadores definem quais elementos da ontologia eles desejam exportar, podendo escolher entre uma exportação completa ou seleções específicas.

2. **Conversão de Dados:** O módulo aplica algoritmos de conversão para transformar os dados ontológicos no formato escolhido, adaptando-os conforme as especificações de cada tipo de arquivo de destino.

3. **Armazenamento e Exportação:** Os dados convertidos são armazenados temporariamente e depois exportados para o local especificado pelo utilizador.

Aplicações práticas O *OBOSData* tem ampla aplicabilidade em diversas áreas, incluindo:

- **Integração de Sistemas:** Facilita a integração de ontologias com sistemas de gestão de dados corporativos, CRMs, e outros sistemas analíticos.

- **Desenvolvimento Web:** Suporta o desenvolvimento de aplicações web que requerem a ingestão e manipulação de dados ontológicos.

- **Ciência de Dados:** Permite que cientistas de dados exportem e utilizem dados ontológicos em ferramentas de análise e machine learning.

O *OBOSData* é, portanto, uma ferramenta essencial dentro do sistema OBOS, oferecendo robustas funcionalidades de gestão e exportação de dados que são fundamentais para o uso eficaz de ontologias em uma variedade de contextos tecnológicos e empresariais.

5.3 Edição

O Editor do sistema OBOS é uma interface crucial e multifuncional projetada para oferecer aos utilizadores a capacidade de manipular, modificar e aperfeiçoar ontologias de maneira intuitiva e eficiente. Esta secção explora os diversos aspetos e ferramentas do Editor do OBOS, destacando como facilita a edição detalhada de ontologias, desde a criação de novos elementos até à revisão e ajuste de estruturas existentes. Através do Editor, o OBOS proporciona aos utilizadores recursos avançados para a edição de classes, propriedades e indivíduos, permitindo adaptações rápidas à medida que novas informações se tornam disponíveis ou quando são necessárias mudanças para alinhar a ontologia com objetivos de pesquisa ou requisitos operacionais específicos. Este componente é essencial para manter a relevância e a precisão dos dados ontológicos num ambiente dinâmico e em constante evolução.

5.3.1 Módulos implementados

No coração do sistema OBOS, uma variedade de módulos implementados desempenha funções cruciais que ampliam e enriquecem a operacionalidade e a utilidade da plataforma. Cada módulo é cuidadosamente projetado para responder a necessidades específicas dentro do ecossistema OBOS, proporcionando ferramentas especializadas que permitem a manipulação, análise, visualização e edição de ontologias. Esta secção destaca os módulos chave que foram implementados no sistema, explicando as suas funcionalidades, o valor que adicionam ao OBOS, e como cada um contribui para transformar ontologias complexas em estruturas acessíveis e manipuláveis. Os módulos implementados são essenciais para a realização das capacidades de alto nível do OBOS, garantindo que os utilizadores possam efetivamente interagir com a plataforma de maneira que responda às suas necessidade específicas dos dados e processamento. Esta introdução aos módulos implementados serve como uma porta de entrada para compreender a versatilidade e a profundidade que o OBOS oferece a seus utilizadores.

OBOSVim

O módulo *OBOSVim* é uma ferramenta integrada ao sistema OBOS que adapta funcionalidades do popular editor de texto Vim para a edição de ontologias. Este módulo é especialmente desenhado para utilizadores que preferem um ambiente de edição baseado em texto e estão familiarizados com os comandos e a interface do Vim. O *OBOSVim* oferece uma série de funcionalidades poderosas que tornam a edição de ontologias eficiente e flexível.

5.3.2 OBOSVim

O módulo *OBOSVim* no sistema OBOS é uma adaptação inovadora do editor de texto Vim para a edição de ontologias. Este módulo combina a eficiência do Vim, conhecido por sua poderosa edição baseada em texto, com as necessidades específicas de manipulação de dados ontológicos. O *OBOSVim* permite aos utilizadores editar, consultar e gerir ontologias utilizando uma interface familiar aos utilizadores experientes do Vim.

Funcionalidades do OBOSVim As principais funcionalidades do *OBOSVim* incluem:

- **Comandos de Edição Vim:** Utiliza os comandos padrões do Vim para edição de texto, o que permite aos utilizadores realizar alterações complexas em ontologias de forma rápida e eficiente.

- **Highlight de Sintaxe:** Suporta *highlight* de sintaxe específico para ontologias, melhorando a legibilidade e facilitando a identificação de classes, propriedades e outros elementos ontológicos.
- **Templates Personalizados:** O módulo utiliza *templates* personalizados que estruturam a visualização e edição de ontologias de maneira lógica e acessível, baseando-se na sintaxe e na estrutura de dados específicas das ontologias.

Template utilizado O *template* utilizado no *OBOSVim* é desenhado especificamente para melhorar a interação com dados ontológicos. Este *template* organiza as informações em uma estrutura clara e acessível, com secções delineadas para classes, propriedades e indivíduos. A implementação do *template* facilita a visualização de relações e hierarquias dentro da ontologia, e é configurável para responder às preferências de visualização do utilizador.

Processo de operação O funcionamento do *OBOSVim* é detalhado nos seguintes passos:

1. **Carregamento da Ontologia:** Ao iniciar o módulo, a ontologia é carregada no editor utilizando o *template* predefinido.
2. **Edição e Consulta:** Os utilizadores podem editar diretamente os dados da ontologia usando comandos do Vim, com suporte para desfazer/refazer, procura e substituição, e outras funcionalidades avançadas de edição.
3. **Guardar e Exportar:** Após a edição, as alterações podem ser guardadas diretamente no sistema operativo ou exportadas para outros formatos compatíveis.

Aplicações práticas O *OBOSVim* é ideal para utilizadores que precisam de uma ferramenta poderosa e flexível para a edição detalhada de ontologias. É particularmente útil em ambientes de pesquisa e desenvolvimento onde são necessárias mudanças frequentes e precisas nas ontologias.

Em resumo, o *OBOSVim* traz a funcionalidade e a flexibilidade do Vim para o mundo das ontologias, proporcionando uma poderosa ferramenta de edição que é adaptada para atender às exigências complexas de manipulação de dados ontológicos.

5.4 Validação

Esta secção descreve os comandos de validação implementados no sistema OBOS, de modo a detalhar as suas funcionalidades específicas. Estes comandos são essenciais para garantir que as ontologias

carregadas e manipuladas no sistema estejam corretas e livres de erros, contribuindo de forma direta para a eficácia e a fiabilidade do sistema.

5.4.1 OBOSLint: Verificação da integridade da ontologia

O OBOSLint é um comando desenvolvido para realizar uma série de verificações de validação em ontologias carregadas no sistema. Este comando automatiza a deteção de erros comuns e garante a conformidade com os padrões de ontologia.

Função e importância do OBOSLint O comando OBOSLint analisa uma ontologia em busca de inconsistências, erros de sintaxe, e violações de regras lógicas que podem comprometer a qualidade e a funcionalidade dos dados semânticos. A ferramenta é projetada para ser altamente configurável, permitindo aos utilizadores especificar quais tipos de verificações devem ser realizadas, baseando-se nas necessidades específicas das suas aplicações.

Características e capacidades As principais características do OBOSLint incluem:

- **Validação sintática:** Verifica se a ontologia está em conformidade com os padrões de formatação e sintaxe do RDF, OWL, ou quaisquer outras linguagens de ontologia utilizadas.
- **Verificação da consistência lógica:** Utiliza inferências e regras lógicas para identificar contradições ou anomalias lógicas dentro da ontologia, garantindo que todos os dados sejam logicamente consistentes.
- **Deteção de erros semânticos:** Procura por erros comuns que possam afetar a interpretação semântica dos dados, como relações mal definidas ou classificações erradas de entidades.

Processo de verificação Ao executar o OBOSLint, o sistema realiza uma série de passos iterativos que incluem a leitura da ontologia, a aplicação de uma série de testes de validação e a geração de um relatório detalhado de quaisquer problemas encontrados. Este relatório pode então ser usado para refinar e aprimorar a ontologia, garantindo que ela atenda aos requisitos de qualidade e precisão necessários para operações críticas.

Aplicações Práticas O OBOSLint é vital para programadores e investigadores que trabalham com ontologias, pois oferece uma ferramenta eficaz para manutenção regular e garantia de qualidade. É particu-

larmente útil em ambientes onde as ontologias precisam ser frequentemente atualizadas ou modificadas, o que ajuda a manter a integridade dos dados ao longo de mudanças e evoluções do sistema.

5.4.2 Comando de testes de regressão

Este comando executa uma bateria de testes automatizados após cada atualização do sistema ou modificação na ontologia, garantindo que as novas alterações não introduzam erros ou problemas em funcionalidades já existentes.

5.4.3 Comando de análise de qualidade de dados

Avalia a qualidade dos dados representados nas ontologias, focando em aspetos como precisão, completude e relevância dos dados. Este comando é vital para manter a alta qualidade dos dados no sistema.

5.4.4 Impacto dos comandos de validação

Os comandos de validação desempenham um papel fundamental na manutenção da integridade e da utilidade das ontologias dentro do OBOS. Assim é possível garantir que os dados e as operações sejam consistentes, precisas e confiáveis, facilitando o uso eficaz do sistema em diversas aplicações.

Capítulo 6

Exemplos

Neste capítulo, são fornecidos exemplos práticos de como o OBOS pode ser utilizado em situações reais. Através de estudos de caso e cenários aplicados, é possíver ver como o sistema operativo pode ser utilizado para resolver problemas específicos no domínio das ontologias.

6.1 OBOSHtml

Serão apresentados alguns *outputs* e demonstrações da capacidade do OBOSHtml. Como este comando inclui funcionalidades do OBOSClasses, essas serão apresentadas aqui.

6.1.1 Árvore de classes

Tomemos de exemplo o excerto da seguinte ontologia:

```
:Avo rdfs:subClassOf :Pai .
:Tiago_Manuel_Queirós_Barata a :Pai .
:António_Manuel_de_Almeida_Barata a :Pai .
:António_Manuel_de_Almeida_Barata a :Avo .
:Maria_de_Nazaré_Queirós_Dias a :Mae .
:Maria_de_Nazaré_Queirós_Dias a :Avo .
```

Listing 6.1: Ontologia exemplo

O output gerado na árvore de classes é o seguinte:

```
thing (12, 12)
    Mae (1,1)
    Pai (3, 5)
        Avo (2, 2)
```

Listing 6.2: Taxonomia OBOSHtml

Podemos ainda gerar a um diagrama de classes através do *graphviz*

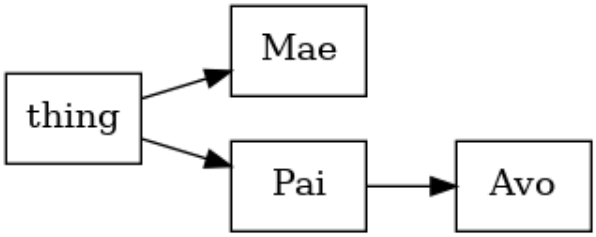


Figura 9: Diagrama de hierarquia de classes

O exemplo acima é meramente ilustrativo. é possível ver um caso real da ontologia FYT no anexo A.

É ainda possível escolher a profundidade máxima da árvore, ou seja o número de subclasses a apresentar:

```
oboshtml -m 1 <ontologia>
```

Listing 6.3: Comando OBOS para escolher a profundidade

Desta forma, o resultado obtido é:

```
thing (12, 12)
    Mae (1,1)
    Pai (3, 5)
```

Listing 6.4: Taxonomia de classes com definição da profundidade

É ainda possível selecionar a(s) classe(s) que queremos mostrar

```
oboshtml -t pai <ontologia>
```

Listing 6.5: Comando OBOS para escolher o topo da árvore de classes

Com este comando, a árvore de classes tem este aspeto

```
Pai (3, 5)
    Avo (2, 2)
```

Listing 6.6: Taxonomia de classes com definição do topo

Os exemplos anteriores funcionam igualmente na geração do diagrama de hierarquia de classes.

6.1.2 Classes

A representação das classes é feita através da cor azul e com algumas especificidades

Avo (2, 2)

subclasse de : Pai

tipo
António Manuel de Almeida Barata
Maria de Nazaré Queirós Dias

Mae (1, 1)

tipo : Maria de Nazaré Queirós Dias

Pai (3, 5)

subclasse de : Avo
tipo
Tiago Manuel Queirós Barata
António Manuel de Almeida Barata

Figura 10: Representação das classes no HTML

As classes aparecem no topo, logo depois da taxonomia de classes com hiperligações para as respetivas classes. Além disso, aparecem ordenadas alfabeticamente, com as respetiva cardinalidade. Dentro da classe, representadas a negrito, consegue-se ver as relações.

Cada classe tem 2 partes, separadas por uma linha cinza horizontal:

1. Triplos na qual é representada como **sujeito**

2. Triplos na qual é representada como **objeto**

No 1º caso, a leitura é intuitiva: classe $\rightarrow$ relação $\rightarrow$ objeto.

No 2º caso, a leitura é feita ao contrário, tendo a seta a fazer essa indicação: objeto $\rightarrow$ relação $\rightarrow$ classe.

No caso de serem feitas inferências através o *inverseOf*, é priorizada a relação em que a classe se mantenha a sujeito, e descartada a outra. Por exemplo, acrescentar-mos mais um triplo à ontologia anterior:

```
rdf:type owl:inverseOf :instância .
```

Listing 6.7: Ontologia exemplo com campo *inverseOf*

Podemos observar de novo o *output* gerado:

Avo (2, 2)

instância
António Manuel de Almeida Barata
Maria de Nazaré Queirós Dias
subclasse de : Pai

Mae (1, 1)

instância : Maria de Nazaré Queirós Dias

Pai (3, 5)

instância
Tiago Manuel Queirós Barata
António Manuel de Almeida Barata

---

subclasse de : Avo ⤴

Figura 11: Representação das classes no HTML com o *inverseOf*

Neste exemplo, tanto na classe `Avo` como na classe `Pai` conseguimos ver diferenças relativas à Representação das classes no HTML. Os indivíduos que anteriormente apareciam no segundo formato ou seja, de leitura ao contrário, agora aparecem no primeiro formato.

6.1.3 Indivíduos

A representação dos indivíduos é feita através da cor vermelha e com algumas especificidades. Utilizando ainda a ontologia anterior, podemos observar os indivíduos.

António Manuel de Almeida Barata

tipo
Pai
Avo

Maria de Nazaré Queirós Dias

tipo
Mae
Avo

Tiago Manuel Queirós Barata

tipo : Pai

Figura 12: Representação dos indivíduos no HTML

Os indivíduos aparecem logo depois das classes. Estão ordenados alfabeticamente e todos eles estão hiperligados com as classes e vice-versa. Toda a estratégia de representação dos indivíduos é semelhante à da classe.

Literais multimédia

É ainda possível adicionar conteúdo multimédia ao HTML. Para isso, basta criar uma pasta chamada "media" com os conteúdos lá dentro. Na ontologia, basta acrescentar as linhas correspondentes que identificam os tipos dos literais:

```
:Tiago_Manuel_Queirós_Barata :musica_favorita "nome da cançao.mp3"^^:mid .
:Tiago_Manuel_Queirós_Barata :retrato "my_foto.png"^^:img .
```

Listing 6.8: Ontologia exemplo com conteúdo multimédia

Após a geração do HTML podemos observar o seguinte:

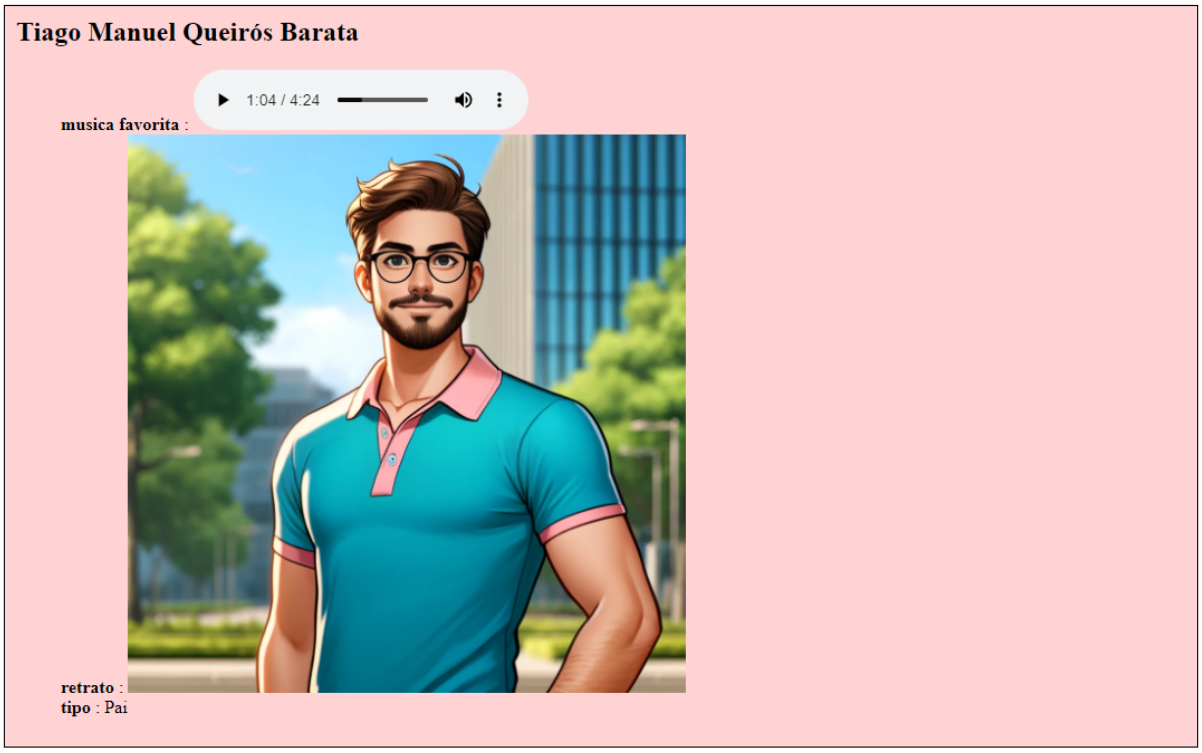


Figura 13: Conteúdo multimédia no HTML

Quando é escrita a definição do tipo, esta é interpretada pelo OBOS e escrita a tag correta no HTML.

Literais textuais

Utilizando agora a ontologia do Eurico Thomaz de Lima como exemplo, podemos ver algumas coisas novas e mais dinâmicas no HTML, nomeadamente:

```
:C999 :tem_como_transcrição  """
    <envelope>
        <envPart side="front">
            <address type="receiver">
                <addrLine><abbr expand="Excelentíssimo">Ex<sup>mo</sup></
                    abbr> <abbr expand="Senhor">Snr</abbr> <lb/>
                <persName>Eurico Tomaz de Lima</persName></addrLine><lb/>
                <addrLine>Academia Mozart</addrLine><lb/>
                <addrLine>Rua do Breynner, 112</addrLine><lb/>
                <addrLine><name type="city">Porto</name></addrLine>
            </address>
        </envPart>

        <envPart side="back">
```

```
14            <address type="sender">
15                <addrLine>Da casa de: <persName>Tomaz de Lima</persName></
                    addrLine>
16                <addrLine><abbr expand="Rua">R</abbr> Luciano Cordeiro 74
                    3<sup>o</sup></addrLine><lb/>
17                <addrLine><name type="city">Lisboa</name></addrLine>
18            </address>
19        </envPart>
20    </envelope>
21    (...)"""^^:tei
```

Listing 6.9: Excerto em TEI da ontologia ETL

Neste caso a ontologia tem um literal com a transcrição de uma carta. Esta em particular é bastante extensa, por isso apresento apenas um excerto. Essa transcrição encontra-se com as normas TEI o que permite alguma formatação adicional, obtendo o seguinte resultado:

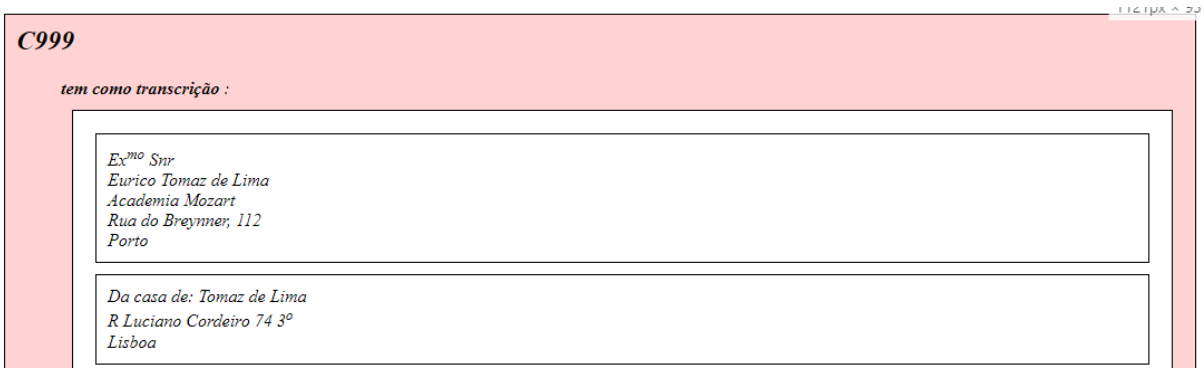


Figura 14: Representação dos indivíduos no HTML com TEI

Todos os ficheiros baseados em XML podem ter um processamento deste género, uma vez que as etiquetas têm uma integração direta com CSS.

Além destes tipos de ficheiros, é possível utilizar outros formatos já pré-formatados, fazendo a sua conversão através de ferramentas próprias, integradas no OBOS:

```
1 :Lacyr_Schettino :biografia """
2 # Lacyr Annunziata Schettino
3
4 (*1914,+2004)
5
6   *  nasceu em Mar de Espanha, Minas Gerais e
7   *  formou-se em Letras e Canto pela PUC-MG.
8   *  Foi professora de teoria e história da música em Sete Lagoas (MG)
```

```
  *  e em Barra Mansa (RJ).
  *  Membro da Academia Mineira de Letras.
  *  Estreou com o livro de poesias Quando as sombras se espalham (1949).

Lacyr Annaziata Schettino, filha do coronel Nunziato Schettino e de
Maria Rosa Schettino, nasceu em Mar de Espanha, Minas Gerais, em 1914,
e faleceu em Belo Horizonte, em 26 de abril de 2004. Sua obra engloba
poesias, pesquisas históricas, traduções, crítica literária e
literatura infantil.
"""
```

Listing 6.10: Excerto em MD da ontologia ETL

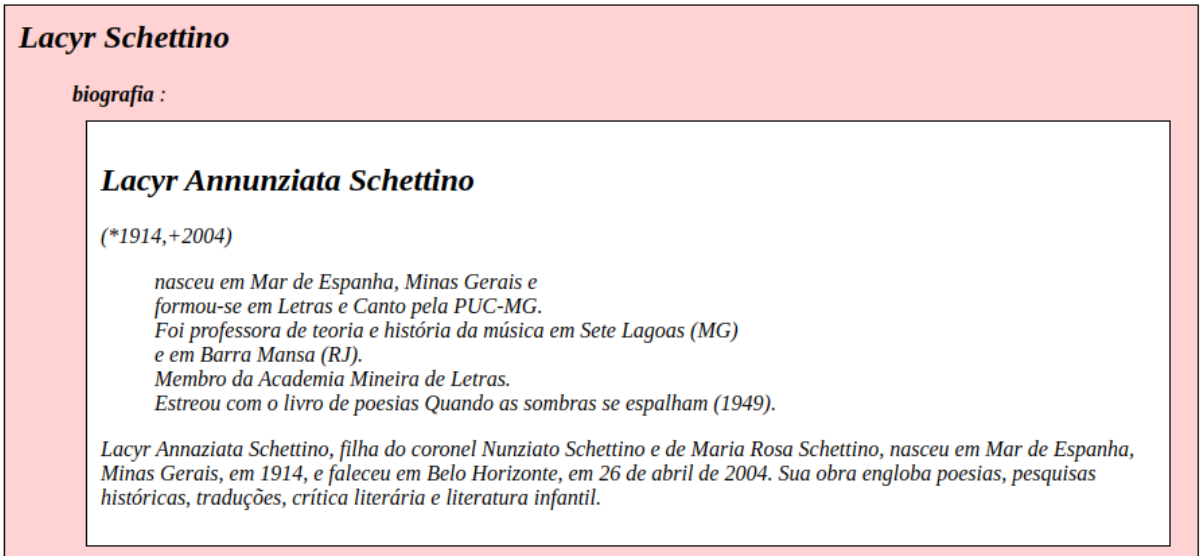


Figura 15: Representação de MD em HTML

6.1.4 Propriedades

Utilizando novamente a ontologia inicial simples [6.1.1], podemos observar as propriedades.

objectProperties

instância
inverso de: tipo
inverso de
subclasse de
tipo
inverso de: instância

dataProperties

Figura 16: Representação das propriedades no HTML

Neste exemplo, podemos observar as propriedades da ontologia, incluindo os atributos dessas propriedades, nomeadamente o inverso, mas poderíamos ter comentário ou range.

6.2 OBOSSignatures

Utilizando a ontologia anteriormente definida 6.1.1, já com todas as alterações efetuadas nas secções seguintes, podemos ver várias saídas possíveis.

```
[Object Properties]
rdf:type
owl:inverseOf
rdfs:subClassOf
:instância

[Data Properties]
:musica_favorita
:retrato
```

Listing 6.11: *Output default* do 5.1.3

Além do *output default*, é possível obter o *output* em JSON:

```
{
  "owl:DatatypeProperty": {
    ":musica_favorita": {},
    ":retrato": {}
  },
  "owl:ObjectProperty": {
    "rdfs:subClassOf": {},
```

```
8     "rdf:type": {
9       "owl:inverseOf": ":instância"
10    },
11    "owl:inverseOf": {},
12    ":instância": {
13      "owl:inverseOf": "rdf:type"
14    }
15  }
16 }
```

Listing 6.12: *Output* em JSON

Podemos ainda obter a saída em YAML:

```
1 owl:DatatypeProperty:
2   :instância:
3     owl:inverseOf: rdf:type
4   owl:inverseOf: {}
5   rdf:type:
6     owl:inverseOf: :instância
7   rdfs:subClassOf: {}
8 owl:ObjectProperty:
9   :musica_favorita: {}
10  :retrato: {}
```

Listing 6.13: *Output* em YAML

Podemos obter o *output* em *turtle*, que pode ser bastante útil para construção de outras ontologias:

```
1 @prefix owl: <http://www.w3.org/2002/07/owl#> .
2 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
3 @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
4
5 <http://obos#instância> a owl:ObjectProperty .
6 <http://obos#musica_favorita> a owl:DatatypeProperty .
7 <http://obos#retrato> a owl:DatatypeProperty .
8
9 rdf:type a owl:ObjectProperty .
10 rdfs:subClassOf a owl:ObjectProperty .
11 owl:inverseOf a owl:ObjectProperty .
```

Listing 6.14: *Output* em *Turtle*

6.3 OBOSTemplate

O comando OBOSTemplate é o comando mais personalizável do OBOS. Esse comando tem a versatilidade de podermos construir um *template* baseado nos nossos dados ontológicos e desta forma criarmos qualquer tipo de documento baseado nas ontologias.

Tomamos a ontologia definida em B.1 como base. É uma ontologia onde é especificada uma pessoa, as suas relações familiares durante a vida, bem como algumas histórias de vida. É ainda colocada uma foto da pessoa.

Neste caso, construi-se o *template* B.2, que constrói uma árvore genealógica, bem como coloca a foto da pessoa, juntamente com as histórias. Na segunda página, após terem sido apresentadas todas as informações relevantes, é apresentada a restante informação irrelevante.

Para isto faz-se uso dos filtros Jinja2 definidos pelo OBOS. Realçam-se os casos seguintes:

- Linha 7 - *relfun* - função que permite obter um objeto relacionado com *p*, com a relação "nome".
- Linha 18 - *relchain* - função que permite obter um predicado relacionado com *p*, mas que não seja a primeira relação. Neste caso, estamos à procura do "género" do "pai" de *p*.
- Linha 20 - *relchain com atributos* - semelhante ao anterior, mas pode levar atributos antes ou depois, nomeadamente strings. Fica uma escrita mais limpa do que estar a concatenar strings no *template*.
- Linha 28 - *rel* - semelhante ao *relfun*, mas retorna uma lista de objetos, em vez de apenas um.
- Linha 78 - *triplosExcept* - função que permite obter todos os triplos relacionados com *p* à exceção dos definidos como argumentos.

Após a criação do *template* é gerado um resultado no mesmo formato. Após usar usar um compilador latex, o resultado final pode ser visto em B.3.

6.4 OBOSGrep

O comando OBOSGrep tem como objetivo fazer pesquisas na ontologia. Se na ontologia anteriormente definida 6.1.1 se fizer uma pesquisa por "avo", obtém-se o seguinte resultado:

```
:António_Manuel_de_Almeida_Barata
    rdf:type
        :Avo

:Avo
    rdfs:subClassOf
        :Pai

:Maria_de_Nazaré_Queirós_Dias
    rdf:type
        :Avo
```

Listing 6.15: Resultado da pesquisa por "avo"

Neste exemplo em concreto, podemos observar que a posição onde o termo de pesquisa aparece, não é relevante. No entanto, a ferramenta está preparada para fazer procuras em pontos específicos dos triplos. Vamos fazer a mesma pesquisa, mas especificando que o resultado só pode aparecer como sujeito:

```
:Avo
    rdfs:subClassOf
        :Pai
```

Listing 6.16: Resultado da pesquisa por "avo" como sujeito

Se neste exemplo em concreto, for feita a pesquisa pela relação, o resultado será vazio.

6.5 OBOSVim

Muito semelhante ao exemplo anterior, mas com a possibilidade de abrir um editor de texto, o 5.3.2 permite o utilizador acrescentar, eliminar ou editar triplos já existentes. Utilizando a pesquisa feita anteriormente 6.4, vai-se fazer a seguinte edição através do vim:

```
:Avo
    rdfs:subClassOf
                :Pai
:Maria_de_Nazaré_Queirós_Dias
    rdf:type
        :Avó
:Artur
    rdf:type
        :Avô
```

Listing 6.17: Edição dos triplos de "avo"

Foi então apagado o triplo relativo ao "António", acrescentado um triplo sobre o "Artur" e ainda acrescentado um acento no tipo da "Maria". A resposta da aplicação foi a seguinte:

```
Nº de triplos adicionados:  2
Nº de triplos apagados:  2
Result:  _EXP_eu.n3
```

Listing 6.18: *Output* da edição

Examinando então o ficheiro `_EXP_eu.n3`, que é a ontologia resultante:

```
@prefix : <http://obos#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:António_Manuel_de_Almeida_Barata a :Pai .
:Artur a :Avô .
:Avo rdfs:subClassOf :Pai .
:Maria_de_Nazaré_Queirós_Dias a :Avó,
        :Mae .
:Tiago_Manuel_Queirós_Barata a :Pai ;
    :musica_favorita "nome da cançao.mp3"^^:mid ;
    :retrato "my_foto.png"^^:img .
rdf:type owl:inverseOf :instância .
```

Listing 6.19: Ontologia editada

Foi assim possível fazer uma edição mais cirúrgica, e mais simples para o utilizador.

Capítulo 7

Conclusões e Trabalho Futuro

7.1 Conclusões

Este documento apresentou uma análise detalhada dos módulos implementados no sistema OBOS, destacando as suas capacidades fundamentais e o papel que desempenham na facilitação da manipulação, análise e visualização de ontologias. Através da implementação de módulos como *obosquery*, *obosvisual*, e *obosdata*, o OBOS demonstrou ser uma plataforma robusta que não suporta apenas a complexidade inerente às ontologias, mas também simplifica essas complexidades para os utilizadores finais.

Os módulos de edição e tradução, particularmente, ofereceram métodos intuitivos e eficazes para interagir com dados ontológicos, permitindo que os utilizadores de diversos níveis técnicos modifiquem, consultem e explorem informações ontológicas com eficiência. A integração dessas ferramentas no OBOS ajudou a estabelecer uma sólida infraestrutura para a gestão de conhecimento que é escalável, flexível e altamente adaptável às necessidades dos utilizadores.

A abordagem modular do sistema permitiu uma personalização extensa e a possibilidade de expandir ou refinar funcionalidades conforme necessidades emergentes. Este sistema provou ser não apenas um recurso valioso para os programadores e investigadores.

7.2 Trabalho futuro

Em primeira instância, o trabalho daqui em diante consistirá em aprimorar o OBOS numa ferramenta ainda mais versátil, mediante uma gestão mais eficiente de possíveis erros. Adicionalmente, será necessário trabalhar na conclusão de todas as ferramentas enunciadas anteriormente.

No futuro, há várias áreas em que o sistema OBOS pode ser expandido e melhorado. Uma das principais áreas de desenvolvimento futuro é a integração de capacidades de inteligência artificial e aprendi-

zagem de máquina para automatizar e otimizar a análise de dados ontológicos. Implementar algoritmos que podem aprender com as interações dos utilizadores e sugerir edições ou melhorias nas ontologias pode aumentar significativamente a eficácia do sistema.

Além disso, há um potencial considerável para desenvolver uma interface de utilizador mais intuitiva e visualmente atraente que poderá tornar o OBOS mais acessível a um público mais amplo. Simplificar a interface enquanto mantém as suas funcionalidades base será essencial para atrair novos utilizadores que podem não ter experiência técnica profunda em ontologias.

Em conclusão, enquanto o sistema OBOS já é uma ferramenta poderosa, as oportunidades para expansão e melhoria são abundantes. O futuro do OBOS está na sua capacidade de se adaptar, evoluir e responder às mudanças nas tecnologias de informação e às necessidades dos utilizadores.

Bibliografia

[1] Clav - classificação e avaliação da informação pública. https://clav.dglab.gov.pt/. Accessed: January 15, 2022.

[2] Wikidata. https://www.wikidata.org/. Accessed: January 15, 2022.

[3] Harith Alani, Yannis Kalfoglou, Kieron O'Hara, and Nigel Shadbolt. Towards a killer app for the semantic web. *Proceedings of the 4th International Semantic Web Conference*, pages 829–843, 2003.

[4] Dean Allemang and James Hendler. *Semantic Web for the Working Ontologist: Effective Modeling in RDFS and OWL*. Elsevier, 2011.

[5] Carla Ambrósio, Filipe Freitas, and Pedro R Henriques. Descrição arquivística em ontologias: Uma proposta para a representação do espólio de eurico thomaz de lima. In *I Encontro Português de Arquivos, Bibliotecas e Centros de Documentação Musicais*, pages 41–49, 2013.

[6] Grigoris Antoniou and Frank van Harmelen. *A Semantic Web Primer*. MIT Press, 2004.

[7] Sören Auer and Christian Bizer. Dbpedia: A nucleus for a web of open data. *The Semantic Web*, 4825:722–735, 2007.

[8] Sean Bechhofer, Rafael Gonçalves, Matthew Horridge, and Luigi Iannone. Why linked data is not enough for scientists. *Future Generation Computer Systems*, 29(2):599–611, 2009.

[9] Dave Beckett. Turtle - terse rdf triple language, 2008. URL https://www.w3.org/TR/turtle/.

[10] Tim Berners-Lee. Weaving the web, 1999.

[11] Tim Berners-Lee. Linked data, 2006. URL https://www.w3.org/DesignIssues/LinkedData.html.

[12] Tim Berners-Lee, James Hendler, and Ora Lassila. The semantic web. *Scientific American*, 2001.

[13] Tim Berners-Lee, James Hendler, and Ora Lassila. The semantic web. *Scientific American*, 284(5): 34–43, 2001.

[14] Michael Bishop. *Learning SPARQL: Querying and Updating with SPARQL 1.1*. O'Reilly Media, 2011.

[15] Jeremy J. Carroll et al. Jena: Implementing the semantic web recommendations, 2011. URL https://jena.apache.org.

[16] Tom Heath Christian Bizer and Tim Berners-Lee. Linked data - the story so far. *International Journal on Semantic Web and Information Systems*, 5(3):1–22, 2009.

[17] James Clark. Xsl transformations (xslt) version 1.0. *W3C Recommendation*, 1999.

[18] Oscar Corcho, Mariano Fernández-López, and Asunción Gómez-Pérez. Methodologies, tools and languages for building ontologies. where is their meeting point? *Data Knowledge Engineering*, 46 (1):41–64, 2003.

[19] MarkLogic Corporation. *MarkLogic Server: Scalable, High-Performance Graph Database*. MarkLogic, 2015.

[20] Mathieu d'Aquin and Enrico Motta. Watson, more than a semantic web search engine. In *Semantic Web Conference*, 2008.

[21] RDFLib Developers. Rdflib github repository. https://github.com/RDFLib/rdflib, 2023.

[22] Bob DuCharme. *Learning SPARQL*. O'Reilly Media, Inc., 2013.

[23] Orri Erling and Ivan Mikhailov. Rdf support in the virtuoso dbms. 2009.

[24] Sean Falconer and Margaret-Anne Storey. Understanding and improving the visualization of hierarchies with treemaps: An empirical study. *Journal of Visual Languages & Computing*, 22(1):1–21, 2011.

[25] Lee Feigenbaum. Sparqling conversations: Rdf access to information through conversation. *Communications of the ACM*, 50(7):45–50, 2007.

[26] Free Software Foundation. *GNU Grep Manual*. GNU Project, 2021. URL https://www.gnu.org/software/grep/manual/grep.html. Accessed: 2024-04-29.

[27] Free Software Foundation. *GNU Coreutils WC Manual*. GNU Project, 2021. URL https://www.gnu.org/software/coreutils/manual/html_node/wc-invocation.html. Accessed: 2024-04-29.

[28] Asunción Gomez-Perez, Mariano Fernandez-Lopez, and Oscar Corcho. *Ontological Engineering*. Springer, 2004.

[29] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[30] Nicola Guarino. Formal ontology and information systems. In *Proceedings of FOIS'98*, 1998.

[31] Andy Harris. *SPARQL 1.1 Query Language*. W3C Recommendation, 2013.

[32] Tom Heath and Christian Bizer. *Linked Data: Evolving the Web into a Global Data Space*. Synthesis Lectures on the Semantic Web: Theory and Technology. Morgan & Claypool, 2011.

[33] James Hendler. Agents and the semantic web. *IEEE Intelligent Systems*, 16(2):30–37, 2002.

[34] Pascal Hitzler, Markus Krötzsch, Bijan Parsia, Peter F. Patel-Schneider, and Sebastian Rudolph. Owl 2 web ontology language primer. W3C Recommendation, October 2009. URL https://www.w3.org/TR/owl2-primer/.

[35] Pascal Hitzler, Markus Krötzsch, Sebastian Rudolph, and York Sure. *Semantic Web: Grundlagen*. Springer, 2010.

[36] Ian Horrocks, Peter F. Patel-Schneider, and Frank van Harmelen. From shiq and rdf to owl: The making of a web ontology language. *Journal of Web Semantics*, 2003.

[37] Eero Hyvönen. Publishing and using cultural heritage linked data on the semantic web. *Morgan & Claypool*, 2009.

[38] Atanas Kiryakov, Damyan Ognyanov, and Dimitar Manov. Owlim: A pragmatic semantic repository for owl. *Web Semantics: Science, Services and Agents on the World Wide Web*, 2(3):182–192, 2005.

[39] Graham Klyne and Jeremy J. Carroll. Resource description framework (rdf): Concepts and abstract syntax, 2004.

[40] Graham Klyne and Jeremy J. Carroll. Resource description framework (rdf): Concepts and abstract syntax, 2006. URL https://www.w3.org/TR/rdf-concepts/.

[41] Holger Knublauch et al. The protégé owl plugin: An open development environment for semantic web applications. In *Third International Semantic Web Conference*, 2004.

[42] Jean-Baptiste Lamy. Owlready2 documentation. https://owlready2.readthedocs.io/en/latest/, 2023.

[43] Ora Lassila and Ralph R. Swick. Resource description framework (rdf) model and syntax specification. *W3C Recommendation*, 1999.

[44] John MacFarlane. Pandoc: A universal document converter, 2013. URL https://pandoc.org.

[45] Deborah L. McGuinness and Frank van Harmelen. Owl web ontology language overview. *W3C Recommendation*, 2003. URL https://www.w3.org/TR/owl-features/.

[46] Noah Mendelsohn. The architecture of the world wide web, volume one. *W3C Recommendation*, 2009.

[47] Brad Miller and David Ranum. *Python Programming in Context*. Jones & Bartlett Learning, 2004.

[48] Bram Moolenaar. *Vim User Manual*. Vim Documentation Project, 2021. URL https://www.vim.org/docs.php. Accessed: 2024-04-29.

[49] Boris Motik, Peter F. Patel-Schneider, and Bijan Parsia. Owl 2 web ontology language: Structural specification and functional-style syntax. *W3C Recommendation*, 2009.

[50] Mark A. Musen. The protégé project: A look back and a look forward. *AI Matters*, 1(4):4–12, 2012.

[51] Mark A. Musen. The protégé project: A look back and a look forward. *AI Matters*, 1(4):4–12, 2015.

[52] Isaac Newton. Isaac newton letter to robert hooke. 1675.

[53] Natalya F. Noy. Semantic integration: A survey of ontology-based approaches. *SIGMOD Record*, 33 (4):65–70, 2004.

[54] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. Technical Report KSL-01-05, Stanford Knowledge Systems Laboratory, 2001.

[55] Natalya F. Noy et al. Creating semantic web contents with protégé-2000. *IEEE Intelligent Systems*, 16(2):60–71, 2006.

[56] Pandoc. Pandoc - about pandoc, 2024. URL https://pandoc.org/.

[57] Peter F. Patel-Schneider, Patrick Hayes, and Ian Horrocks. Owl web ontology language semantics and abstract syntax. *W3C Recommendation*, 2004.

[58] Peter F. Patel-Schneider, Patrick Hayes, and Ian Horrocks. Owl web ontology language semantics and abstract syntax. *W3C Recommendation*, 2004.

[59] Valentina Presutti and Eva Blomqvist. Identity of resources and entities in the semantic web. *Journal of Web Semantics*, 2013.

[60] Pallets Projects. Jinja2 documentation. https://jinja.palletsprojects.com/, 2023.

[61] Eric Prud'hommeaux and Andy Seaborne. Sparql query language for rdf. W3C Recommendation, 2008. URL https://www.w3.org/TR/rdf-sparql-query/.

[62] Python-Markdown Project. Python-markdown, 2024. URL https://python-markdown.github.io/.

[63] Python Software Foundation. *re — Regular expression operations*, 2024. URL https://docs.python.org/3/library/re.html.

[64] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. Semantics and complexity of sparql. *ACM Transactions on Database Systems*, 34(3):16, 2006.

[65] Eric S. Raymond. *The Art of Unix Programming*. Addison-Wesley Professional, 2003.

[66] Alan Rector. Modularisation of domain ontologies implemented in description logics and related formalisms including owl. *Knowledge Engineering Review*, 2003.

[67] Tânia Alexandra Costa Rego. Ontologia eurico thomaz de lima: Representação de conhecimento a partir da correspondência do espólio, 2022.

[68] Dennis M. Ritchie and Ken Thompson. The unix time-sharing system. *Communications of the ACM*, 17(7):365–375, 1974.

[69] Nigel Shadbolt, Wendy Hall, and Tim Berners-Lee. *The Semantic Web Revisited*, volume 21. 2006.

[70] Pavel Shvaiko and Jérôme Euzenat. A survey of schema-based matching approaches. *Journal on Data Semantics IV*, pages 146–171, 2005.

[71] Ray Siemens and John Unsworth. *A New Companion to Digital Humanities*. Wiley-Blackwell, 2016.

[72] Barry Smith. Ontology. In *Blackwell Guide to the Philosophy of Computing and Information*, pages 155–166. Blackwell, 2003.

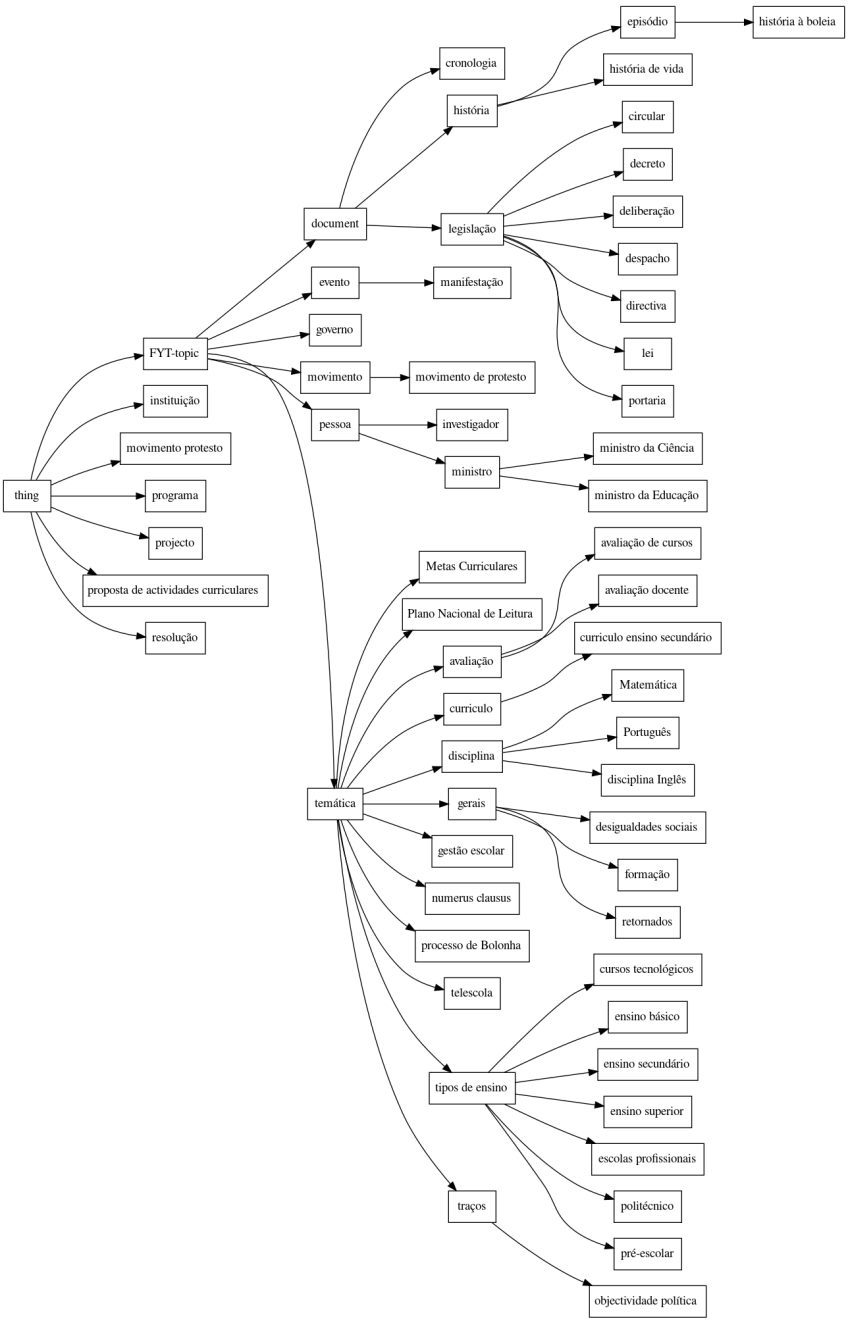
[73] Steffen Staab and Rudi Studer. *Handbook on Ontologies*. Springer, 2009.

[74] RDFLib Team. Rdflib documentation. https://rdflib.readthedocs.io/en/stable/, 2023.

[75] Mike Thompson et al. Bigdata: A distributed architecture for rdf data management. *Databases*, 2014.

Apêndice A
Diagrama de classes ontologia FYT

Neste capítulo será apresentado o diagrama de classes de um caso real da ontologia FYT.



Apêndice B
Uso de filtros ontológicos em templates

Neste capítulo pode observar-se uma ontologia simples sobre uma pessoa, e as suas relações familiares. Com base nesta ontologia, foi criado um *template* ontológico que utiliza filtros jinja2 implementados no OBOS. Após a geração, pode-se observar o resultado.

B.1 Ontologia

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xml: <http://www.w3.org/XML/1998/namespace> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
#@prefix : <t#> .

:irmao a owl:ObjectProperty .
:pai a owl:ObjectProperty .
:mae a owl:ObjectProperty .
:genero a owl:ObjectProperty .
:HasSpouse a owl:ObjectProperty .
:filho a owl:ObjectProperty .
:historia a owl:ObjectProperty .
:amigo a owl:ObjectProperty .
:ex a owl:ObjectProperty .
:sobre a owl:ObjectProperty .
:body a owl:ObjectProperty .
:morreu a owl:ObjectProperty .

:nasceu a owl:DatatypeProperty .
:nome a owl:DatatypeProperty .
:img a owl:DatatypeProperty .
:titulo a owl:DatatypeProperty .

:Pessoa_A a :Pessoa .
:Pessoa_A :irmao :Pessoa_B .
:Pessoa_A :pai :Pessoa_C .
:Pessoa_A :mae :Pessoa_D .
:Pessoa_A :nasceu "1998-05-11"^^xsd:date .
:Pessoa_A :nome "Pedro"^^xsd:string .
:Pessoa_A :genero :male .
:Pessoa_A :HasSpouse :Pessoa_E .
:Pessoa_A :filho :Pessoa_F .
:Pessoa_A :filho :Pessoa_G .
:Pessoa_A :filho :Pessoa_H .
```

```
:Pessoa_A :historia :Uma_Aventura_no_Gerês .
:Pessoa_A :historia :Minas_dos_Carris .
:Pessoa_A :img "myFoto.png"^^:img .
:Pessoa_A :amigo :Pessoa_I .
:Pessoa_A :amigo :Pessoa_J .
:Pessoa_A :amigo :Pessoa_K .
:Pessoa_A :ex :Pessoa_L .
:Pessoa_A :ex :Pessoa_M .
:Pessoa_A :ex :Pessoa_N .
:Pessoa_A :ex :Pessoa_O .
:Pessoa_A :ex :Pessoa_P .

:Pessoa_C a :Pessoa .
:Pessoa_C :nasceu "1974-07-22"^^xsd:date .
:Pessoa_C :nome "António"^^xsd:string .
:Pessoa_C :genero :male .

:Pessoa_D a :Pessoa .
:Pessoa_D :nasceu "1969-06-23"^^xsd:date .
:Pessoa_D :nome "Josefa"^^xsd:string .
:Pessoa_D :genero :female .

:Pessoa_B a :Pessoa .
:Pessoa_B :nasceu "2006-03-21"^^xsd:date .
:Pessoa_B :nome "Sofia"^^xsd:string .
:Pessoa_B :genero :female .

:Pessoa_E a :Pessoa .
:Pessoa_E :nasceu "1999-09-10"^^xsd:date .
:Pessoa_E :nome "Maria"^^xsd:string .
:Pessoa_E :genero :female .

:Pessoa_F a :Pessoa .
:Pessoa_F :nasceu "2030-02-19"^^xsd:date .
:Pessoa_F :nome "João"^^xsd:string .
:Pessoa_F :genero :male .

:Pessoa_G a :Pessoa .
:Pessoa_G :nasceu "2028-02-19"^^xsd:date .
:Pessoa_G :nome "António Jr"^^xsd:string .
:Pessoa_G :genero :male .

:Pessoa_H a :Pessoa .
:Pessoa_H :nasceu "2033-02-19"^^xsd:date .
:Pessoa_H :nome "Maria João"^^xsd:string .
:Pessoa_H :genero :female .

:Uma_Aventura_no_Gerês a :historia .
:Uma_Aventura_no_Gerês :titulo "Uma aventura no Gerês"^^xsd:string .
:Uma_Aventura_no_Gerês :sobre :Pessoa_A .
:Uma_Aventura_no_Gerês :sobre :Pessoa_E .
:Uma_Aventura_no_Gerês :body """Certo dia, estavamos a passear no gerês e
    apareceu um lobo. Sim, isso mesmo, um lobo!! Nós começamos a fugir, e
    escondemo-nos... quando demos por ela era a toca do lobo, mas está tudo
    bem, entretanto ficamos amigos e fizemos um churrasco."""^^xsd:string .

:Minas_dos_Carris a :historia .
:Minas_dos_Carris :titulo "As Minas dos Carris são um perigo"^^xsd:string .
```

```
:Minas_dos_Carris :sobre :Pessoa_A .
:Minas_dos_Carris :sobre :Pessoa_F .
:Minas_dos_Carris :sobre :Pessoa_G  .
:Minas_dos_Carris :sobre :Pessoa_H .
:Minas_dos_Carris :body """Uma vez fomos fazer uma caminhada às minas dos
    Carris, e começou mega nevão... só parou quando saiu uma notícia a dizer
     que estávamos perdidos, A verdade é que não estávamos, e quando
    chegamos cá baixo até oferecemos ajuda à polícia.
Apercebemo-nos entretanto que aquele aparato todo em busca de uns
    caminhantes perdidos era à nossa procura ahaha
Coitados :)"""^^xsd:string .
```

Listing B.1: Ontologia exemplo

B.2 Template

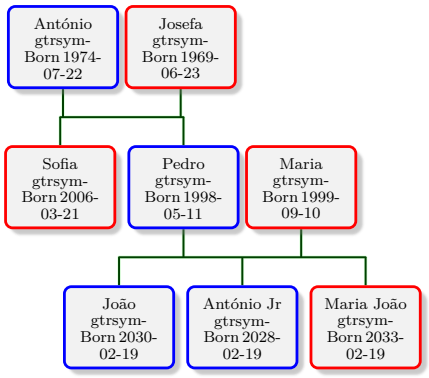
```
\documentclass{article}
\usepackage[top=4.5cm, bottom=4.5cm, left=2cm, right=2cm]{geometry}
\usepackage[all]{genealogytree}
\usepackage[export]{adjustbox}
\usepackage{blindtext}

\title{\textbf{ {{ p | relfun('nome') }} }}

\begin{document}
\maketitle

\noindent
\begin{minipage}{.5\textwidth}
    \begin{tikzpicture}
        \genealogytree[template=signpost, id suffix=@p]
        {
         child{
          p[{{ p | relchain('pai','genero') }}]{
            {{ p | relchain('pai', 'nome') }}
            {{ p | relchain('pai', 'nasceu', preText='\\\gtrsymBorn\,') }}
            {{ p | relchain('pai', 'morreu', preText='\\\gtrsymDied\,') }}
          }
          p[{{ p | relchain('mae','genero') }}]{
            {{ p | relchain('mae', 'nome') }}
            {{ p | relchain('mae', 'nasceu', preText='\\\gtrsymBorn\,') }}
            {{ p | relchain('mae', 'morreu', preText='\\\gtrsymDied\,') }}
          }
            {% for elem in p | rel('irmao') %}
                c[{{ elem | relfun('genero') }}]{
                    {{ elem | relfun('nome') }}
                    {{ elem | relfun ('nasceu', preText='\\\gtrsymBorn\,')
                        }}
                    {{ elem | relfun ('morreu', preText='\\\gtrsymDied\,')
                        }}
                }
            {% endfor %}

           child{

            g[{{ p | relfun('genero') }}]{
```

```
                {{ p | relfun('nome') }}
                {{ p | relfun ('nasceu', preText='\\\gtrsymBorn\,') }}
                {{ p | relfun ('morreu', preText='\\\gtrsymDied\,') }}
            }
            p[{{ p | relchain('HasSpouse', 'genero') }}]{
                {{ p | relchain('HasSpouse', 'nome') }}
                {{ p | relchain('HasSpouse', 'nasceu', preText='\\\gtrsymBorn\,') }}
                {{ p | relchain('HasSpouse', 'morreu', preText='\\\gtrsymDied\,') }}
            }
            {% for elem in p | rel('filho') %}
                c[{{ elem | relfun('genero') }}]{
                    {{ elem | relfun('nome') }}
                    {{ elem | relfun ('nasceu', preText='\\\gtrsymBorn\,') }}
                    {{ elem | relfun ('morreu', preText='\\\gtrsymDied\,') }}
                }
            {% endfor %}
           }
         }
        }
    \end{tikzpicture}
\end{minipage}
\begin{minipage}{.5\textwidth}
     \includegraphics[width=.6\textwidth, right]{
            {{ p | relfun('img') }}
        }
\end{minipage}% This must go next to `\end{minipage}`

{% for elem in p | rel('historia') %}
    \section*{
        {{ elem | relfun('titulo') }}
    }
    {{ elem | relfun('body') }}
{% endfor %}

\section*{Outra info}
\subsection*{
    {{ p | relfun('nome') }}
}
\begin{itemize}
    {% for key, val in p | triplosExept('nome', 'img','nasceu', 'morreu', 'pai', 'mae', 'irmao', 'HasSpouse', 'filho', 'genero', 'historia') | dictsort %}
        \item {{ key }}
        \begin{itemize}
            {% for elem in val | sort %}
                \item {{ elem }}
            {% endfor %}
        \end{itemize}
    {% endfor %}
\end{itemize}

\end{document}
```

Listing B.2: Template exemplo

B.3 Resultado

Pedro



Uma aventura no Gerês

Certo dia, estavamos a passear no gerês e apareceu um lobo. Sim, isso mesmo, um lobo!! Nós começamos a fugir, e escondemo-nos... quando demos por ela era a toca do lobo, mas está tudo bem, entretanto ficamos amigos e fizemos um churrasco.

As Minas dos Carris são um perigo

Uma vez fomos fazer uma caminhada às minas dos Carris, e começou mega nevão... só parou quando saiu uma notícia a dizer que estávamos perdidos, A verdade é que não estávamos, e quando chegamos cá baixo até oferecemos ajuda à polícia. Apercebemo-nos entretanto que aquele aparato todo em busca de uns caminhantes perdidos era à nossa procura ahaha Coitados :')

1

Outra info

Pedro

- amigo
  - Pessoa I
  - Pessoa J
  - Pessoa K
- ex
  - Pessoa L
  - Pessoa M
  - Pessoa N
  - Pessoa O
  - Pessoa P
- tipo
  - Pessoa